

Finales resueltos de Criptografía y Seguridad

Exámenes finales (72.44) — resoluciones completas

*Este PDF reúne la resolución de varios exámenes finales de la materia. Cada ejercicio incluye la **consigna textual**, los **temas** involucrados (con referencia a la clase/unidad de los resúmenes de la materia), **qué necesitás saber** y la **resolución completa y razonada**, construida exclusivamente a partir del material de la materia (resúmenes de Primera y Segunda Parte).*

Instituto Tecnológico de Buenos Aires (ITBA)

Índice

Introducción	2
1. Final — 21/07/2021	3
1.1. Ejercicio 1 — Licitación transparente con AES-GCM (Tecnociudad)	3
1.2. Ejercicio 2 — Elección de supervisor mediante hash y XOR	5
1.3. Ejercicio 3 — Pentesting con hipótesis de falla (Agrolin)	6
2. Final — 17/07/2023	9
2.1. Ejercicio 1 — Dispositivos de medición de agua (AES-ECB, HTTPS, integridad)	9
2.2. Ejercicio 2 — Ocultar un mensaje “tipo sombras” (OTP + integridad con hash/H-MAC)	11
2.3. Ejercicio 3 — Pentesting a una página de noticias (hipótesis de falla)	13
3. Final — 06/07/2023	15
3.1. Ejercicio 1 — k -anonimato en base de datos de emails (SPAM)	15
3.2. Ejercicio 2 — Red de sensores de oleoducto: CHACHA20 + HMAC-SHA3	17
3.3. Ejercicio 3 — Hipótesis de falla para pentest de app móvil de exchange	19
4. Final — 10/07/2023	22
4.1. Ejercicio 1 — Sensor $\rightarrow$ maestro con AES-CBC: integridad y replay	22
4.2. Ejercicio 2 — Alternativas de MAC para datos grandes (árbol de Merkle)	24
4.3. Ejercicio 3 — Vulnerabilidades de un sitio de noticias	26
5. Final — 2025, Segundo Cuatrimestre (1)	30
5.1. Ejercicio 1 — Pulseras Bluetooth (AES-CTR, modo degradado)	30
5.2. Ejercicio 2 — Protocolo de autenticación Cliente-Servidor ($H(K_c r)$)	32
5.3. Ejercicio 3 — CasaIA (IoT hogareño): hipótesis de falla	34
6. Ejercicios adicionales	36
6.1. Ejercicio — Bingo virtual	36
6.2. Ejercicio — Robo de cables eléctricos	38
6.3. Ejercicio — Organizame (pentesting de una app de calendario)	41

Introducción

Este documento resuelve los ejercicios de los siguientes exámenes finales de Criptografía y Seguridad (72.44): **21/07/2021, 17/07/2023, 06/07/2023, 10/07/2023, 2025 — C2 (1)**, y un conjunto de **ejercicios adicionales** (bingo virtual, sensores de robo de cable, app “Organízame”). Por cada ejercicio se muestra la consigna textual, los temas que toca citando la unidad/clase correspondiente de los resúmenes de la materia (Primera Parte: criptografía; Segunda Parte: seguridad), y una resolución completa.

Regla de oro seguida en toda resolución: no se agrega nada que no esté en el material de la materia (resúmenes de Primera y Segunda Parte). Cuando algún punto de un enunciado excede ese material, se lo señala explícitamente en la resolución en lugar de inventar contenido.

Nota sobre las fuentes de los finales: varios de los PDF de los exámenes ya traían respuestas manuscritas de un alumno rendido. Esas respuestas se usaron solo como referencia de qué se preguntaba; cada resolución de este documento fue re-elaborada y verificada contra los resúmenes de la materia, seleccionando lo correcto y corrigiendo o completando lo que no se ajustaba al material de la materia.

1 Final — 21/07/2021

1.1 Ejercicio 1 — Licitación transparente con AES-GCM (Tecnociudad)

Consigna (textual)

El consejo deliberante del partido de Tecnociudad solicitó la construcción de un sistema distribuido que permita realizar licitaciones de forma transparente y segura. Cuando el partido necesita contratar cualquier tipo de servicios, genera un pliego con la descripción de lo solicitado y espera que distintos contratistas presenten ofertas. Durante esta fase, los costos son secretos para evitar que un oferente tenga preferencia. Una vez vencido el plazo, se publica la propuesta de cada uno, junto con el precio, y se selecciona el que cumple con las condiciones con el menor costo.

La implementación del sistema asegura que estas propiedades ocurran utilizando criptografía. La descripción a alto nivel es la siguiente:

- Se publica la licitación con un ID único asociado.
 - Por cada licitación, se reciben archivos cifrados con AES-GCM de parte de los oferentes interesados. Los archivos incluyen un documento con el ID de la licitación, la descripción técnica, el CUIT de la empresa y el precio.
 - Al terminar el periodo de la licitación los oferentes comparten la clave utilizada, y con ello se revisan todas las ofertas, y se adjudica la ganadora.
 - Si un oferente no provee la clave, queda automáticamente descalificado.
 - Para transparencia, se publican todas las ofertas recibidas ya descifradas y la adjudicada en un portal público que puede ser consultado libremente.
- a) Explicar por qué ningún oferente puede alterar su propuesta sin ser detectado. (1 pto.)
 - b) Qué medidas adicionales serían necesarias para evitar que algún administrador del sistema ignore a un proveedor diciendo que nunca entregó la clave. (1 pto.)
 - c) Considerar el caso de un oferente que envía un archivo a nombre de un competidor, con un precio irrisoriamente bajo que lo perjudicaría. ¿Cómo evitaría esto usando funciones criptográficas? (1 pto.)

Temas. Primera Parte, Clase 3 — MACs y Cifrado Auténtico (§Modos AEAD en la práctica: GCM; §Definición de cifrado auténtico; §No-repudio: por qué los MAC no lo dan). Primera Parte, Clase 4 — Asímetro y Firma Digital (§Diferencia entre cifrar y firmar; §RSA-Signature / firma del hash; §PKI y certificados digitales, no-repudio).

Qué necesitás saber.

- **AES-GCM** es un modo AEAD (*cifrado auténtico*): además de confidencialidad (modo Counter), produce una etiqueta de autenticación sobre el **texto cifrado**. Al descifrar, primero se verifica el tag; si el cifrado fue modificado, la verificación **falla** y el descifrado se rechaza (no se obtiene “basura” silenciosa como con cifrado no auténtico).
- **Encrypt-then-MAC / AEAD** da **confidencialidad + integridad + autenticación**, pero **con clave simétrica compartida NO da no-repudio**: quien verifica el tag/GHASH conoce la misma clave que quien lo generó, así que un tercero no puede distinguir quién produjo el mensaje (ambas partes podrían haberlo generado).

- Para **no-repudio** hace falta criptografía **asimétrica: firmar** (con la clave **privada** del firmante) es lo que da no-repudio, porque solo el dueño de la privada pudo producir esa firma, y cualquiera la verifica con la pública sin poder falsificarla.

Notas y conexiones. El escenario combina dos capas que el material distingue con cuidado: (i) AES-GCM protege la *confidencialidad e integridad del archivo cifrado en tránsito/reposo* durante la licitación (nadie puede leer los precios antes de tiempo ni modificar un archivo ya enviado sin que se note), pero (ii) por ser **simétrico y con clave compartida**, no resuelve el problema de que un administrador (que también conoce/gestiona las claves) niegue haber recibido algo, ni impide que alguien se haga pasar por otro oferente. Ambos puntos requieren la capa de **firma digital / PKI** de la Clase 4, que sí da autenticación de origen y no-repudio.

Resolución. a) Ningún oferente puede alterar su propuesta sin ser detectado porque el archivo se envía cifrado con **AES-GCM**, que es un esquema de **cifrado autenticado** (AEAD): además de cifrar el contenido (confidencialidad), GCM calcula una **etiqueta de autenticación** sobre el texto cifrado (usando GHASH). Al momento de descifrar (una vez compartida la clave), se **verifica primero el tag**; si el archivo cifrado fue modificado en cualquier bit respecto del que se envió originalmente, la verificación del tag **falla** y el descifrado se rechaza explícitamente (a diferencia de un cifrado sin autenticar, donde la manipulación pasaría desapercibida y solo produciría datos “basura” al descifrar). Por lo tanto, cualquier intento de alterar el precio o la descripción técnica *después* de enviado el archivo es detectable al momento de abrir las ofertas.

b) El esquema descrito usa una **clave simétrica compartida** (el oferente “comparte la clave utilizada”); como se remarca en la Clase 3, un esquema con clave compartida (MAC/AEAD simétrico) **no da no-repudio**: quien controla el sistema (el administrador) también podría — al menos en principio — estar en condiciones de negar la recepción de la clave sin que el oferente pueda probar lo contrario, ya que no hay una firma verificable por un tercero independiente. La medida adicional es introducir **firma digital (asimétrica)**: que el oferente, al enviar la clave de descifrado, la **firmo con su clave privada**, y que el administrador, al recibirla, devuelva un **acuse de recibo firmado** con su propia clave privada. De esta forma:

- La firma del oferente sobre el envío de la clave prueba, ante cualquier tercero (verificando con la clave pública del oferente), que efectivamente la envió.
- El acuse de recibo firmado por el administrador (con su clave privada, verificable con su pública conocida vía PKI/certificado) es una prueba de que el sistema la recibió; como solo el administrador pudo generar esa firma, **no puede negarlo después** (no-repudio).

Esto traslada el problema de “palabra contra palabra” a un esquema con **propiedades de no-repudio**, que el cifrado simétrico por sí solo no ofrece.

c) El problema es que el sistema, tal como está descrito, no verifica que quien envía un archivo “a nombre de” un CUIT sea realmente el dueño de esa identidad: cualquiera podría escribir el CUIT de un competidor dentro del documento cifrado. Esto se evita usando **firma digital**: cada oferente debe **firmar su propuesta** (el documento completo: ID de licitación, descripción técnica, CUIT y precio) con su **clave privada** antes de cifrarlo y enviarlo, y esa clave privada debe estar asociada a su identidad mediante un **certificado digital** emitido por una **CA** (PKI), tal como en el ejemplo de firma de binarios de la Clase 4. Al adjudicar, el administrador (o cualquiera con la clave pública certificada del oferente) **verifica la firma**: si el documento dice ser del competidor pero está firmado con la clave privada del atacante, la verificación con la clave pública *del competidor* **falla**, y la propuesta fraudulenta se descarta. En síntesis: **AES-GCM** da confidencialidad e integridad del archivo, pero la **autenticidad del emisor** (que el CUIT declarado sea quien realmente presentó la oferta) requiere **firma digital con PKI**, que es justamente lo que impide la suplantación de identidad descripta.

1.2 Ejercicio 2 — Elección de supervisor mediante hash y XOR

Consigna (textual)

El siguiente protocolo fue propuesto para que N nodos de un sistema puedan elegir un supervisor, de tal manera que ningún nodo pueda ganar o perder sistemáticamente manipulando el intercambio:

1. Cada nodo genera un valor r_i de 64 bits y envía al resto $H(r_i)$.
2. Luego de recibir todos los valores, cada nodo publica r_i .
3. Cada nodo calcula $r_x = r_1 \oplus r_2 \oplus \dots \oplus r_n$.

El nodo a menor distancia de r_x es el nuevo supervisor, utilizando la distancia de Hamming como métrica (se comparan bit a bit los dígitos binarios de los dos valores, y cada discrepancia suma 1 de distancia — por ejemplo 0010 y 1110 están a distancia 2). En caso de empate, se reinicia.

- a) Explicar por qué el paso 1 es necesario y no se puede comenzar el intercambio en el paso 2 directamente. (1 pto.)
- b) Considerar el caso de solo tres nodos. ¿Qué pasaría si el nodo 2, por ejemplo, decide abusar el sistema, y repite en el paso 1 el valor $H(r_1)$ como si fuese el suyo? Proponer una modificación simple que evite el problema. (1,5 pts.)
- c) Proponer una mejora al algoritmo para que no sea necesario repetir todo el intercambio en caso de empate y siempre se llegue a un resultado que no pueda ser manipulado. (0.5 pts.)

Temas. Primera Parte, Clase 3 — MACs y Cifrado Autenticado (§Funciones de hash: resistencia a preimagen, segunda preimagen y colisiones; uso como *compromiso*/commitment previo a revelar un valor). **Nota:** el protocolo en sí (elección de supervisor con XOR y distancia de Hamming) no está descripto literalmente en el material como esquema con nombre propio; la resolución se apoya en las **propiedades de las funciones de hash** que sí están desarrolladas en la Clase 3, aplicadas como un esquema de compromiso (committing a r_i vía $H(r_i)$ antes de revelarlo).

Qué necesitás saber.

- **Resistencia a preimagen:** dado un hash y , es computacionalmente inviable hallar un mensaje x tal que $H(x) = y$. Este es el uso relevante acá: publicar $H(r_i)$ primero *compromete* al nodo con un valor sin revelarlo, porque nadie puede, a partir del hash, deducir r_i para ajustar su propio valor en función de los demás.
- **Resistencia a segunda preimagen:** dado un mensaje x , es inviable hallar $x' \neq x$ con el mismo hash. Esto es lo que impide, una vez publicado $H(r_i)$, cambiar el r_i real por otro r'_i que también hashee a lo mismo en el paso 2.
- El material señala explícitamente el uso de compromisos criptográficos (“probar que se tenía algo antes de revelarlo”; “juegos de cartas con compromiso”) como aplicación de la resistencia a (segunda) preimagen.

Notas y conexiones. El paso 1 (publicar $H(r_i)$ antes de r_i) es exactamente el patrón de **compromiso-y-revelación** (commit-then-reveal) que la Clase 3 asocia a la resistencia a preimagen/segunda preimagen: sirve para que ningún participante pueda elegir su valor *después* de ver

los de los demás (lo cual rompería la imparcialidad de la elección). Es el mismo principio que sustenta, por ejemplo, no poder generar de antemano el r_i en función del resultado final.

Resolución. a) El paso 1 (publicar $H(r_i)$ antes de r_i) es necesario porque, si en cambio se comenzara directamente publicando los r_i en claro (paso 2), un nodo malicioso podría esperar a ver los valores r_j ($j \neq i$) que ya publicaron todos los demás y **recién entonces** elegir su propio r_i de forma tal que el XOR resultante r_x quede convenientemente cerca (en distancia de Hamming) de su propio valor, garantizándose ser supervisor. Publicar primero $H(r_i)$ actúa como un **compromiso**: por **resistencia a preimagen**, ningún otro nodo puede, a partir de $H(r_i)$, obtener r_i antes de que se revele en el paso 2, de modo que nadie puede ajustar su propio valor en función de los r_i de los demás. Así se garantiza que todos los r_i quedaron “congelados” antes de que ninguno conociera el de los otros, haciendo el intercambio imparcial.

b) Con tres nodos, si el nodo 2 en el paso 1 publica $H(z) = H(r_1)$ (repite el compromiso del nodo 1) en lugar de comprometerse con su propio valor, en el paso 2 puede publicar $z = r_1$ como si fuera su r_2 (es un valor válido porque su hash coincide con lo publicado). El resultado sería $r_x = r_1 \oplus r_1 \oplus r_3 = r_3$ (el XOR de r_1 consigo mismo se cancela), y el nodo 2 conocería de antemano que $r_x = r_3$: puede entonces mirar la distancia de Hamming entre r_1, r_2^{real} y r_3 contra $r_x = r_3$ y decidir manipular el resultado a su favor (por ejemplo, el propio nodo 3 quedaría siempre exactamente a distancia 0 de r_x , y el nodo 2 puede jugar con esa certeza, o directamente evitar que el nodo 1 o 3 ganen si le conviniera). En cualquier caso, se rompe la propiedad buscada de que “ningún nodo pueda ganar o perder sistemáticamente manipulando el intercambio”.

Modificación simple: exigir que **todos los compromisos $H(r_i)$ publicados en el paso 1 sean distintos entre sí** ($H(r_i) \neq H(r_j)$ para todo $i \neq j$); si dos nodos publican el mismo hash, se detecta la colisión de compromisos y se reinicia el proceso (o se descalifica al nodo repetido). Como por **resistencia a segunda preimagen** es inviable para el nodo 2 encontrar un $r'_2 \neq r_1$ propio con $H(r'_2) = H(r_1)$, la única forma de publicar un hash igual al de otro nodo es copiándolo literalmente, lo cual esta verificación detecta y bloquea antes de llegar al paso 2.

c) Para no tener que reiniciar todo el intercambio (generar nuevos r_i y repetir los tres pasos) en caso de empate en la distancia de Hamming, se puede definir un **criterio de desempate determinístico y no manipulable** sobre la información que ya está publicada, sin generar nueva aleatoriedad: por ejemplo, entre los nodos empatados en distancia mínima a r_x , elegir como supervisor al que tenga **el valor r_i numéricamente mayor (o menor)** interpretado como entero, o el que primero coincide bit a bit con r_x leyendo de izquierda a derecha. Como los r_i ya fueron comprometidos con hash y revelados (y no pueden cambiarse sin romper la resistencia a segunda preimagen del compromiso), este criterio de desempate no puede ser manipulado por ningún nodo a esta altura del protocolo (todos los valores ya están fijados e inmutables), y siempre produce un único ganador sin necesidad de reiniciar el intercambio completo.

1.3 Ejercicio 3 — Pentesting con hipótesis de falla (Agrolin)

Consigna (textual)

Agrolin S.A. se dedica a la construcción de soluciones tecnológicas para el campo. Recientemente se publicó un informe que menciona múltiples vulnerabilidades en uno de sus productos y provocó muchas pérdidas de ventas a punto de concretarse. Para corregir la situación, la empresa decidió contratar un servicio de *pen testing*, realizar una evaluación más exhaustiva y corregir el producto para empezar a recuperar el daño a su imagen. El producto en cuestión permite hacer un seguimiento del rendimiento de las cosechas a lo largo del tiempo y determinar las rotaciones óptimas para mejorar la productividad. El sistema está compuesto por un frontend web que es un *single page web app*, donde el usuario puede registrarse, autenticarse, dar de alta lotes marcado sus límites en un mapa, actuali-

zarlos o eliminarlos. El sistema muestra según datos históricos de todos los clientes de la zona, el rinde esperado para la superficie en cuestión, y sugiere el tipo de cultivo con mayor rentabilidad en función de los valores de rinde y precios de mercados futuros estimados, y permite que el usuario ingrese los datos del tipo de cultivo y luego de la cosecha el rinde obtenido para comparar y poder mejorar en las próximas sugerencias.

La aplicación web se ejecuta en los navegadores de cada cliente, y hace llamadas a un servidor backend a través de una API REST.

Siguiendo la metodología de hipótesis de falla, establecer 4 hipótesis de vulnerabilidades que tengan alta probabilidad de existir en el escenario descripto. Para cada una, describir únicamente la hipótesis y la prueba que realizaría para confirmarla o refutarla (1 pto. por hipótesis).

Aclaraciones: No se considerarán válidas hipótesis genéricas. Tampoco se considerarán dos hipótesis que sean variantes del mismo tipo de falla (por ejemplo, el mismo problema en dos pantallas distintas).

Temas. Segunda Parte, Clase 6 — Pentesting (§Metodología de Hipótesis de Falla: los pasos; §Paso 2 — Planteo de hipótesis de falla). Segunda Parte, Clase 3 — Principios de Diseño y Vulnerabilidades (§CWE / Top 25 MITRE: SQL Injection CWE-89; §OWASP Top 10; §STRIDE: Denial of Service, Spoofing/Improper Authentication).

Qué necesitás saber.

- La **metodología de hipótesis de falla** exige, en el paso de **planteo de hipótesis**, asumir la existencia de una vulnerabilidad concreta *antes* de probarla; luego se define una **prueba** específica para confirmarla o refutarla (no alcanza con nombrar la categoría genérica: hay que decir *qué* se sospecha y *cómo* se lo comprueba en este sistema puntual).
- El material lista, dentro del **Top 25 CWE de MITRE**, entre las debilidades más comunes: **CWE-89 SQL Injection** (falta de control en el ingreso de datos que permite ejecutar comandos SQL: los datos del usuario se mezclan sin sanitizar con el comando y la base los interpreta como código; defensa: *prepared statements*/ parametrizar), **CWE-79 Cross-site Scripting**, y (Clase 3, taxonomía STRIDE) **Denial of Service** (agotar los recursos necesarios para dar el servicio, afectando disponibilidad) como categoría de amenaza reconocida explícitamente como *ataque de seguridad* aunque no comprometa confidencialidad/integridad.
- **OWASP** publica el Top 10 de vulnerabilidades web más comunes (frameworks de referencia junto con MITRE), relevante para un frontend *single page web app* + API REST como el descripto (autenticación, autorización, control de acceso a recursos por ID).

Notas y conexiones. El enunciado da todas las pistas típicas de una aplicación web con API REST y autenticación de usuarios: (1) el sistema tiene **autenticación** (registrarse/autenticarse), (2) tiene **ingreso de datos de usuario** que probablemente lleguen a una base de datos (lotes, límites en mapa, tipo de cultivo, rinde), (3) hace **consultas agregadas costosas** sobre datos históricos de todos los clientes de la zona, y (4) maneja **recursos por identificador** (lotes) que pertenecen a distintos clientes. Cada una de estas pistas mapea a una categoría de vulnerabilidad del Top 25/CWE o de STRIDE vista en la Clase 3, y la consigna pide aplicar sobre ellas el paso de **hipótesis + prueba** de la Clase 6 (no basta con nombrar la categoría).

Resolución. Hipótesis 1 — Autenticación deficiente (Improper Authentication). *Hipótesis:* el sistema de autenticación del frontend/API es débil o directamente inexistente en algunos endpoints, permitiendo iniciar sesión con credenciales triviales o acceder a funciones que

deberían requerir sesión válida sin ella. *Prueba*: intentar registrarse con credenciales simples (usuario/contraseña predecibles) y verificar si el sistema las acepta sin exigir complejidad; adicionalmente, intentar invocar directamente los endpoints de la API REST (alta/consulta de lotes) sin enviar token/cookie de sesión, para verificar si el backend responde igual que a un usuario autenticado.

Hipótesis 2 — SQL Injection (CWE-89) en el ingreso de datos. *Hipótesis*: los campos donde el usuario ingresa datos (tipo de cultivo, límites del lote, rinde obtenido) no sanitizan/parametrizan la entrada antes de construir la consulta a la base de datos, permitiendo inyectar comandos SQL. *Prueba*: en un campo de texto (por ejemplo, nombre del cultivo), ingresar una cadena con sintaxis SQL (p. ej. un valor que cierre la comilla e intente insertar/alterar una tabla) y verificar, sin necesidad de dañar la base, si el sistema devuelve un error de sintaxis SQL expuesto o un comportamiento anómalo que indique que el dato fue interpretado como comando en lugar de como valor.

Hipótesis 3 — Denial of Service por consultas agregadas costosas. *Hipótesis*: la funcionalidad que muestra, según datos históricos de **todos los clientes de la zona**, el rinde esperado y sugiere el cultivo más rentable en función de precios de mercados futuros, requiere procesar/parsear múltiples consultas pesadas por cada solicitud; enviar varias de estas solicitudes en paralelo podría agotar los recursos del servidor y denegar el servicio a otros usuarios. *Prueba (acotada, sin ejecutar el ataque real)*: verificar únicamente si es posible disparar 2 de estas consultas costosas en paralelo desde una misma sesión/IP sin que el sistema las limite (rate limiting) o las rechace; no se realiza el ataque a mayor escala para no afectar el servicio, y se reporta la situación si se confirma la ausencia de límites.

Hipótesis 4 — Autorización deficiente / control de acceso a recursos de otro cliente (Improper Authorization / Broken Access Control). *Hipótesis*: una vez autenticado, el sistema no verifica correctamente que el usuario solo pueda acceder a sus propios lotes, permitiendo acceder a lotes (y sus límites en el mapa, rindes, etc.) de otros clientes modificando un identificador en la URL o en la llamada a la API REST. *Prueba*: tras crear una cuenta y autenticarse, identificar la URL/endpoint que expone un lote propio con su identificador (p. ej. .../cliente/12/campo/1) y probar acceder al mismo recurso cambiando el identificador de cliente por uno ajeno (p. ej. .../cliente/1/campo/1); si el sistema devuelve los datos sin validar que el lote pertenece al usuario autenticado, se confirma la hipótesis. No se modifican datos ajenos para no ser intrusivo; solo se confirma el acceso indebido.

Nota sobre alcance: las cuatro hipótesis corresponden a categorías distintas de falla (autenticación, inyección de datos, disponibilidad y autorización), tal como exige la aclaración de la consigna, y todas están sustentadas en la taxonomía CWE/STRIDE/OWASP desarrollada en la Clase 3 y en el paso de “planteo de hipótesis” de la metodología de pentesting de la Clase 6.

2 Final — 17/07/2023

2.1 Ejercicio 1 — Dispositivos de medición de agua (AES-ECB, HTTPS, integridad)

Consigna (textual)

Hay una empresa que se encarga de controlar el agua. Para esto la empresa implementa unos dispositivos encargados de hacer las respectivas mediciones. Los mismos le informan a la central los datos recolectados.

El procedimiento cuando se pone un dispositivo es el siguiente: primero el dispositivo emite su ID por broadcast, luego la central responde a ese ID y le envía una llave por HTTP. Cuando el dispositivo le debe enviar lo recolectado a la empresa usa AES-ECB como método de encriptación.

La empresa contrata una auditoría y le dice que posee fallas en la comunicación de la llave y problemas de integridad en la comunicación de los dispositivos.

- Usando HTTPS ¿se hubieran solucionado todos los problemas que informó la auditoría?
- En caso de no poder usar HTTPS por cuestiones de presupuestos ¿qué otro protocolo utilizaría para el intercambio de claves?
- ¿Qué cambios son necesarios para asegurar la integridad en el envío de información de los dispositivos hacia la central?

Temas. Primera Parte, Clase 2 — Cifrado (§ECB, §modos de cifrado, §CPA-security); Clase 3 — MACs y Cifrado Autenticado (§qué es un MAC, §cifrado autenticado, §modos AEAD: CCM/GCM); Clase 5 — Protocolos (§TLS Handshake, §Diffie–Hellman); Segunda Parte, Clase 11 — Protección de Datos (§protocolos de transporte cifrados: TLS/HTTPS).

Qué necesitás saber.

- ECB (Electronic Codebook):** cifra cada bloque de forma independiente, $c_i = e_k(m_i)$. Es determinista: bloques de texto plano iguales producen cifrados iguales $\Rightarrow$ **revela patrones/estructura** y **no es CPA-Secure**. No debe usarse.
- HTTPS = HTTP sobre TLS.** El TLS Handshake acuerda un método de intercambio de clave (RSA, Diffie–Hellman con certificado, ECDH, etc.), autentica con firmas y deriva un *Master Secret*; la criptografía negociable para el resto de la sesión incluye cifrado simétrico (recomendado AES-GCM) y HMAC/AEAD para integridad.
- Diffie–Hellman** (Clase 5): acuerdo de clave cuya seguridad descansa en el problema del logaritmo discreto (DLP); permite que ambas partes compartan una clave secreta sobre un canal inseguro sin haberla intercambiado antes. **DH puro no autentica:** es vulnerable a Man-in-the-Middle si no se combina con certificados (por eso en TLS va con RSA/DSS).
- Cifrado autenticado / AEAD** (Clase 3): en la práctica, si se necesita confidencialidad casi siempre también se necesita integridad, porque el cifrado simple (como ECB) puede ser manipulado sin que nadie lo note. La recomendación es usar un modo **AEAD** como **AES-GCM** (o CCM), que da confidencialidad e integridad juntas.

Notas y conexiones. El escenario combina dos fallas de diseño típicas de la materia: (1) enviar una clave en claro por un canal no cifrado (HTTP) para luego usarla en un cifrado simétrico, y (2) usar ECB, el modo que la Clase 2 marca como “NO utilizar” por no ser CPA-Secure y no dar integridad. Conecta con la distinción Clase 3 entre cifrado (confidencialidad) y MAC/AEAD (integridad): un cifrado por sí solo, aunque fuera un modo seguro como CBC o GCM sin autenticar, no impide que un atacante manipule bits del cifrado sin ser detectado si no hay un mecanismo de integridad de por medio.

Resolución. a) No, no necesariamente se hubieran solucionado todos los problemas.

- Usando **HTTPS con TLS** para el paso donde la central envía la llave, sí se soluciona el problema de la **comunicación de la llave**: TLS resuelve el intercambio de claves de forma autenticada (certificados) y cifra el canal, evitando que un atacante intercepte la llave en texto plano durante el handshake inicial. Esto ataca directamente la falla “fallas en la comunicación de la llave”.
- Sin embargo, HTTPS solo protege la **comunicación** del dispositivo con la central (la llave viajando por HTTP). **No cambia el método de encriptación que el dispositivo usa para enviar lo recolectado**, que sigue siendo **AES-ECB**. Como ECB es determinista y no es CPA-Secure, **no da integridad** (bloques de datos repetidos producen cifrados idénticos, y un atacante puede reordenar/reemplazar bloques de cifrado sin que el receptor lo detecte, ya que ECB no incluye ningún mecanismo de autenticación). Por lo tanto el problema de **integridad en la comunicación de los dispositivos no queda resuelto** solo con HTTPS en el paso de intercambio de la llave.

b) Diffie–Hellman. Si no se puede pagar el costo de HTTPS/TLS completo, se puede usar directamente un protocolo de **intercambio de clave Diffie–Hellman**: permite que la central y el dispositivo acuerden una clave secreta compartida sobre un canal inseguro sin haberla compartido antes, basando su seguridad en el problema del logaritmo discreto. Queda como salvedad (marcada en el material) que **DH puro no autentica a las partes** y es susceptible a un ataque Man-in-the-Middle si un atacante se interpone en el intercambio; para mitigarlo haría falta combinarlo con algún mecanismo de autenticación (certificados o claves públicas ya conocidas), tal como TLS lo hace con RSA/DSS.

c) Para asegurar la integridad en el envío de la información recolectada hacia la central hace falta:

1. **Dejar de usar ECB en solitario**: es un modo sin ningún mecanismo de autenticación, determinista y no CPA-Secure.
2. **Pasar a un esquema de cifrado autenticado (AEAD)**, que da confidencialidad e integridad en una sola construcción: **AES-GCM** (recomendación práctica del material: rápido, estandarizado, en todas las librerías serias) o **AES-CCM**. Ambos agregan, además del cifrado, un *tag* de autenticación que permite al receptor verificar que el mensaje cifrado no fue modificado en el camino.
3. Alternativamente (o en conjunto), agregar explícitamente un **MAC/HMAC** sobre lo enviado, de forma que la central pueda **chequear que el mensaje no fue modificado en el camino** (verificar el tag con la clave compartida antes de aceptar la medición).

Nota de fidelidad: el material no detalla un protocolo específico de bajo costo distinto de Diffie–Hellman para IoT (p. ej. protocolos livianos de sensores); la respuesta se limita a lo cubierto en las clases de Protocolos y MACs/Cifrado Autenticado.

2.2 Ejercicio 2 — Ocultar un mensaje “tipo sombras” (OTP + integridad con hash/HMAC)

Consigna (textual)

Se quiere ocultar un mensaje de forma segura con una técnica similar a las sombras. Dado un mensaje “ m ” se eligen dos números random (x_1, x_2) y se generan por dos canales separados (posiblemente en lugares distintos):

- $p_1 = x_1 \oplus m \oplus x_2$

- $p_2 = x_2 \oplus x_1$

- p_1 y $h(p_2)$

- p_2 y $h(p_1)$

- a) Explicar por qué si un atacante tiene acceso a solo 1 canal de comunicación aun se mantiene la confidencialidad.
- b) Explicar por qué si un atacante tiene acceso a un solo canal de comunicación aun se mantiene la integridad.
- c) Si un atacante tiene acceso a los 2 canales de información, cómo hacer para que se mantenga solo la integridad.

Temas. Primera Parte, Clase 2 — Cifrado (§secreto perfecto, §One-Time Pad / Vernam); Clase 5 — Protocolos (§secreto compartido de Shamir, umbral (t, n) : con menos del umbral no se filtra información); Clase 3 — MACs y Cifrado Autenticado (§funciones de hash, §HMAC, §no-repudio: por qué MAC/hash no lo dan).

Qué necesitás saber.

- **OTP (Clase 2):** $e_k(m) = m \oplus k$ con k uniforme y del mismo tamaño que m tiene **secreto perfecto**: $\Pr[M=m \mid C=c] = \Pr[M=m]$. Si x_1 y x_2 son aleatorios e independientes, $x_1 \oplus x_2$ se comporta como una clave uniforme.
- **Secreto compartido / esquema umbral (Clase 5):** repartir información en “sombras” tal que **con menos de las necesarias no se filtra ninguna información** del secreto (entropía máxima). Acá el mensaje se reparte en dos partes (p_1, p_2) tal que ninguna por separado revela m .
- **Función de hash (Clase 3): no tiene clave** (es información pública); depende de todos los bits del mensaje (efecto avalancha) y permite detectar si el mensaje fue alterado comparando el hash recibido contra el hash recalculado. Al no tener clave, un atacante que **controla el mensaje y puede recalcul el hash él mismo** puede modificar ambos de forma consistente sin ser detectado.
- **HMAC (Clase 3):** MAC construido a partir de una función de hash y una **clave** k compartida; a diferencia del hash simple, el atacante sin la clave **no puede recalcul un tag válido** aunque conozca y modifique el mensaje.

Notas y conexiones. El esquema es análogo a un **secreto compartido de 2 sombras** (Clase 5) combinado con **OTP** (Clase 2): p_1 es el mensaje enmascarado con el XOR de dos

valores aleatorios (equivalente a una clave OTP $x_1 \oplus x_2$), y p_2 es esa misma máscara sin el mensaje. Ninguna de las dos partes por sí sola alcanza para reconstruir m : hace falta juntar ambos canales, igual que con el umbral de Shamir se necesita juntar t sombras. El agregado de $h(\cdot)$ cruzado conecta con Clase 3 (funciones de hash sin clave) y anticipa la necesidad de HMAC para integridad *autenticada* cuando el atacante controla todo.

Resolución. a) Confidencialidad con acceso a un solo canal.

- **Canal 1** entrega $(p_1, h(p_2))$, con $p_1 = x_1 \oplus m \oplus x_2$. Como x_1 y x_2 son elegidos al azar e independientes, $k = x_1 \oplus x_2$ se comporta como una **clave OTP uniforme**: $p_1 = m \oplus k$. Por el resultado de secreto perfecto del OTP, sin conocer k el atacante no obtiene ninguna información sobre m a partir de p_1 solo. $h(p_2)$ tampoco aporta nada sobre m (depende solo de $p_2 = x_1 \oplus x_2$, que no involucra a m).
- **Canal 2** entrega $(p_2, h(p_1))$, con $p_2 = x_1 \oplus x_2$. Esto es exactamente la máscara k usada para enmascarar m , pero **sin el mensaje**: no contiene ninguna información de m . $h(p_1)$ tampoco revela m porque una función de hash no es invertible (no permite recuperar el mensaje original a partir del resumen).

En ambos casos se necesitan **ambos canales** para calcular $p_1 \oplus p_2 = (x_1 \oplus m \oplus x_2) \oplus (x_1 \oplus x_2) = m$; con uno solo la confidencialidad de m se mantiene.

b) Integridad con acceso a un solo canal. Cada canal lleva el valor a transmitir junto con el **hash del otro** valor (p_1 con $h(p_2)$, y p_2 con $h(p_1)$), es decir, hashes **cruzados**. Si el atacante solo controla un canal (por ejemplo el que lleva p_1 y $h(p_2)$) y modifica p_1 en tránsito, no tiene forma de recalcular un $h(p_2)$ consistente porque **no tiene acceso a p_2** (viaja por el otro canal, que no controla). Al llegar los dos valores al receptor, este recalcula $h(p_2)$ recibido en el otro canal y $h(p_1)$ recibido en este, y si p_1 fue alterado, el hash recalculado de p_1 no coincidirá con el $h(p_1)$ que viajó por el canal 2, delatando la alteración. Con un solo canal comprometido, la verificación cruzada contra el otro canal (no comprometido) detecta la manipulación.

c) Con acceso a los dos canales, mantener solo la integridad. Si el atacante controla ambos canales, puede leer p_1 y p_2 enteros, calcular $h(\cdot)$ él mismo (la función de hash **no usa clave**, es pública) y modificar p_1 y/o p_2 recalculando los hashes cruzados de forma consistente: el esquema de hash simple deja de servir para integridad en ese caso (y de hecho la confidencialidad de m también se pierde, porque con los dos canales se recupera $m = p_1 \oplus p_2$; por eso la consigna pide mantener *solo* la integridad, asumiendo que la confidencialidad ya no es sostenible con acceso total a ambos canales).

Para mantener la integridad hay que reemplazar el hash simple $h(\cdot)$ por un mecanismo **que dependa de una clave que el atacante no tenga**:

- Usar **HMAC** en lugar de un hash sin clave: $\text{HMAC}_k(p_2)$ en el canal 1 y $\text{HMAC}_k(p_1)$ en el canal 2, con una clave k secreta compartida entre emisor y receptor mediante un canal previo seguro. Sin conocer k , el atacante no puede generar un tag HMAC_k válido para un p_1 o p_2 modificado, aunque vea y edite el contenido de ambos canales.
- Alternativamente, **firmar digitalmente** p_1 y p_2 con la clave **privada** del emisor: el atacante, aun controlando ambos canales, no puede falsificar la firma sin esa clave privada, por lo que cualquier alteración se detecta al verificar la firma con la clave pública correspondiente. (Esta variante, a diferencia de HMAC, además daría no-repudio, ya que el material remarca que un MAC/HMAC simétrico **no** da no-repudio porque ambas partes comparten la misma clave.)

2.3 Ejercicio 3 — Pentesting a una página de noticias (hipótesis de falla)

Consigna (textual)

Una página de noticias en la que hay un usuario y un generador de noticias. A cada noticia se le puede poner etiquetas como categorías (Deportes, Interés general) o Ubicación (Argentina). Cada usuario lleva un puntaje (+1 por cada noticia rateada positivamente y -1 por cada noticia rateada negativamente). La página tiene una API REST y los llamados a la misma se hacen por HTTPS.

Se le pagará por visualizaciones al generador de la noticia teniendo en cuenta algunos factores que no me acuerdo bien y no venían al caso.

Se hizo un análisis de la página y se encontró que:

1. Existe una URL para pedir todas las noticias:
`https://dominio.com/news?limit=100&page=20`
2. Cuando se le recomienda o no recomienda una noticia se hace a través de `https://dominio.com/{generator_id}/{news_id}/upvoted` o `https://dominio.com/{generator_id}/{news_id}/downvoted`
3. Las personas se registran por usuario y password
4. Se chequearon los inputs de la página y cualquier intento de tag HTML ingresado aparecerá como texto en la página → XSS no probable.

Establecer 4 hipótesis de falla con las respectivas pruebas que haría.

Temas. Segunda Parte, Clase 10b — Pentesting (§Metodología de Hipótesis de Falla: los pasos en orden — Recolección, Hipótesis, Prueba, Generalización, Eliminación; §Injection flaws; §OWASP); Clase 8 — Principios de Diseño y Vulnerabilidades (relacionado: validación de entrada, XSS/CWE-79).

Qué necesitás saber.

- La **metodología de hipótesis de falla** (Clase 10b) tiene 5 pasos en orden: **Recolección de información** → **Planteo de hipótesis de falla** → **Prueba** → **Generalización** → **Eliminación** (opcional). Cada hipótesis debe: (1) partir de la información recolectada del sistema (los 4 hallazgos del enunciado), (2) suponer una vulnerabilidad concreta, y (3) tener una **prueba** asociada que la verifique.
- El material define el **XSS (CWE-79)** como inyección de JS ejecutado por el navegador de la víctima, y aquí el enunciado ya lo descarta explícitamente (los tags HTML se muestran como texto, no se ejecutan) — es un hallazgo del paso de Recolección que cierra esa hipótesis.
- El material menciona en general **OWASP** como organización que publica el Top 10 de vulnerabilidades web y menciona **Injection flaws** (SQLi, XSS, OS command) como la familia más frecuente (~90% según el profesor).

Aviso de fidelidad al material

El enunciado del alumno (usado solo como referencia) etiqueta las hipótesis con nombres de categorías estilo OWASP Top 10 API (“Improper Authentication”, “Broken Access Control”, “BOLA”, ataques de *replay*). Los resúmenes de la materia **sí** cubren OWASP como organización/Top 10 y la familia de *Injection flaws*, y **sí** cubren conceptos de control

de acceso/autenticación de otras clases, pero **no** desarrollan esas siglas puntuales (BOLA, Improper Authentication, Broken Access Control como categorías nombradas) como parte del temario de Pentesting. Por eso, en la resolución de abajo se plantean las 4 hipótesis con el vocabulario y la metodología efectivamente vistos en la Clase 10b (hipótesis de falla + prueba), evitando nombrar categorías OWASP específicas que no están en el material, y describiendo la falla en términos genéricos de control de acceso / autenticación / lógica de negocio ya cubiertos en Segunda Parte.

Resolución. Aplicando la metodología de hipótesis de falla (Clase 10b) sobre los 4 hallazgos de la Recolección de información:

1. **Hipótesis 1 — falta de límite de intentos en el login.** El sistema se registra y autentica por usuario y contraseña (hallazgo 3), y no hay ninguna mención de control sobre los intentos de autenticación. **Hipótesis:** el endpoint de login no limita ni registra intentos fallidos, permitiendo probar contraseñas por fuerza bruta contra una cuenta válida. **Prueba:** realizar múltiples intentos de autenticación contra un usuario existente con contraseñas distintas y observar si el sistema bloquea, demora o registra los intentos repetidos, o si los permite indefinidamente.
2. **Hipótesis 2 — falta de verificación del vínculo usuario/generator_id al votar.** Los endpoints de recomendación usan {generator_id}/{news_id}/upvoted y downvoted directamente en la URL (hallazgo 2), sin mostrar cómo se valida contra quién está autenticado. **Hipótesis:** el back-end no verifica que el generator_id de la URL corresponda al usuario autenticado o autorizado para votar, permitiendo que un usuario vote (sume/reste puntaje) en nombre de otro generador o sin estar autenticado. **Prueba:** (i) sin loguearse, llamar directamente a la URL de upvoted/downvoted con un generator_id/news_id válidos y observar si el sistema restringe el acceso; (ii) logueado como un usuario, llamar a la misma URL con un generator_id que no es el propio y observar si el sistema lo permite.
3. **Hipótesis 3 — manipulación del parámetro limit de la URL de noticias.** El hallazgo 1 expone una URL con parámetros limit y page controlados por el cliente. **Hipótesis:** el valor de limit no está validado ni acotado en el servidor, permitiendo pedir una cantidad arbitrariamente grande de noticias en una sola request (degradación del servicio / consumo excesivo de recursos). **Prueba:** modificar el valor de limit a números extremadamente altos (o negativos/no numéricos) y observar la respuesta y el tiempo/recursos que consume el servidor.
4. **Hipótesis 4 — falta de control sobre votos repetidos del mismo usuario.** El puntaje del generador depende de +1/-1 por cada rating (hallazgo del enunciado principal). **Hipótesis:** el sistema no valida que un mismo usuario ya haya votado una misma noticia, permitiendo repetir el llamado a upvoted/downvoted para inflar o deflar artificialmente el puntaje de un generador (equivalente a un ataque de repetición sobre la misma acción). **Prueba:** repetir la misma llamada de upvoted/downvoted (mismo usuario, misma noticia) dos o más veces y verificar si el puntaje del generador se incrementa cada vez o si el sistema detecta y descarta el duplicado.

Nota: el hallazgo 4 del enunciado (chequeo de tags HTML) queda **descartado** como hipótesis de XSS por el propio enunciado (paso de Recolección ya cerrado): cualquier tag ingresado se muestra como texto, no se ejecuta, así que no aplica la familia de *Injection flaws* tipo XSS sobre ese input en particular.

3 Final — 06/07/2023

3.1 Ejercicio 1 — k -anonimato en base de datos de emails (SPAM)

Consigna (textual)

Considerar una base de datos de emails que han sido utilizados para publicidad no deseada (SPAM) y un servicio que permite a cualquier persona consultar si su email está en la lista. La consulta misma le da al dueño del sistema acceso a nuevos emails que podrían ser vendidos, robados, o utilizados sin consentimiento.

El servicio pretende mantener el anonimato de la información, en el sentido de no poder asociar un email particular a un usuario que usa el mismo. Para ello se decidió implementar un mecanismo conocido como k -anonymity, que establece la imposibilidad de asociar un dato a una entidad discernible en un grupo de al menos k entidades.

El servicio guarda en su base de datos el resultado de aplicar un hash criptográfico SHA-256 sobre cada email. Un cliente que quiera hacer una búsqueda, calcula localmente el hash del email a consultar y envía **solo los primeros x bits** del hash (donde x es una constante definida como parte del servicio), recibiendo como respuesta todos los hashes de la base de datos que tengan dicho prefijo. De esta manera el usuario puede buscar si aparece su hash completo en la respuesta y responder la pregunta.

- ¿Mejora la seguridad si se utiliza un SALT de 256 bits para almacenar los hashes? ¿Tiene alguna otra consecuencia? Aclaración: seguridad en este contexto refiere al modelo a anonimato deseado. (1 pto)
- Explicar por qué alguien que controle el servidor no puede saber ni cuál era el email sobre el que preguntó el usuario, ni si el resultado de la búsqueda fue exitoso o no. (1 pto)
- ¿Con qué criterio se puede definir el valor x (la cantidad de bits de prefijo que debe enviar el cliente) si queremos garantizar 256-anonymity? (1 pto)

Temas. Primera Parte, Clase 3 — MACs y funciones de hash (§Funciones de hash; §Resistencia a preimagen/segunda preimagen/colisión; §Paradoja del cumpleaños). Segunda Parte, Clase 2 — Autenticación (§Salt: qué hace y qué no hace un salt). **Aclaración de fidelidad:** el término “ k -anonimato” / k -anonymity **no aparece** como tal en los resúmenes de la materia (Primera ni Segunda Parte); la definición usada acá es la que trae el propio enunciado del examen. La resolución se arma con las propiedades de funciones de hash y de salt que sí están en el material.

Qué necesitás saber.

- **Resistencia a preimagen:** dado un hash y , es computacionalmente imposible hallar x con $h(x) = y$. Por eso el servidor, recibiendo un prefijo del hash, no puede “invertirlo” para recuperar el email en claro.
- **Salt:** perturbación aleatoria embebida en el hash, definida (según la materia) para el escenario de *cracking offline de contraseñas*: no agranda el espacio de claves del usuario, pero multiplica por $2^{\text{bits de salt}}$ el costo del atacante que intenta adivinar el valor original, porque impide reutilizar un cómputo $h(a)$ contra varias cuentas en paralelo (cada cuenta usa una perturbación distinta).
- **Efecto avalancha / tamaño de salida fijo:** el hash de un mensaje arbitrario da una etiqueta de tamaño fijo (acá 256 bits); dos entradas distintas dan salidas que no guardan relación aparente entre sí.

Notas y conexiones. El protocolo del enunciado es, en esencia, un esquema de **PIR (búsqueda privada) construido ad-hoc** sobre resistencia a preimagen: el cliente nunca envía el email en claro, solo un prefijo de su hash, y el servidor responde con *todos* los hashes que matchean ese prefijo (no solo el buscado) para no revelar cuál interesa puntualmente. Esto conecta con la idea de la materia de que un hash **no es reversible** (“un hash sirve para encriptar” es FALSO, unidad de MACs) y con la distinción salt vs. espacio de claves (unidad de Autenticación): acá el salt no protege contra fuerza bruta sobre el hash sino que **rompe el protocolo mismo**, como se ve en el inciso a).

Resolución. a) **No mejora la seguridad/anonimato buscado; al contrario, rompe el protocolo.** El objetivo del esquema no es dificultar un ataque de fuerza bruta offline contra el hash (que es para lo que el salt sirve en el material: penalizar al atacante que intenta adivinar x tal que $h(x) = y$), sino permitir que el propio usuario legítimo recompute *localmente* el hash de su email para poder consultarlo. Si se agrega un salt s por registro ($h(\text{email}||s)$), el cliente que quiere consultar **no conoce el salt** usado para su propio email en la base (el salt es justamente aleatorio y — para que sirva su propósito de la materia — no se le entrega al usuario de antemano), por lo que no puede recalcular localmente el prefijo correcto para preguntar. Consecuencias:

- Si el servicio de todos modos le entregara el salt correspondiente a su email para que pueda calcular el hash, eso **ya revela al servidor qué salt se pidió**, y si el salt es por-email, pedir “el salt de mi email” equivale a decir qué email se está consultando — **se pierde el anonimato** que se buscaba (el servidor sabe qué registro se está consultando, aunque no vea el email en texto plano).
- Si en cambio se usa **un único salt global** para toda la base, no cambia nada respecto de no usar salt: todos los emails se perturban con el mismo valor, así que el cliente puede seguir calculando el hash localmente sin pedir nada al servidor (el salt global es efectivamente parte de la constante pública del hash).

En síntesis: un salt por-cuenta **rompe el protocolo** (el cliente no puede recalcular el hash sin filtrar información al servidor); un salt global no aporta nada al anonimato porque equivale a no usar salt. En ningún caso mejora el anonimato deseado.

b) El cliente calcula $\text{SHA256}(\text{email})$ **localmente** y envía solo los primeros x bits de ese hash. Por **resistencia a preimagen** de SHA-256, el servidor no puede invertir esos x bits para recuperar el email (ni siquiera el hash completo, del cual solo recibe un prefijo). Además, como el servidor responde con **todos** los hashes completos de la base que comparten ese prefijo (no solo el que interesa), no puede saber cuál de esos hashes de la respuesta es el que realmente le importa al cliente: la comparación final (¿aparece mi hash completo en la lista recibida?) la hace el cliente **del lado suyo**, sin informarle nada al servidor sobre el resultado. Por eso el servidor no puede saber ni qué email se consultó (solo ve un prefijo compartido por muchos emails posibles) ni si la búsqueda fue exitosa (esa comparación ocurre localmente en el cliente, nunca se envía de vuelta).

c) El enunciado define k -anonymity como la imposibilidad de asociar un dato a una entidad discernible en un grupo de **al menos k entidades**. Aplicado a este esquema, se necesita que el prefijo de x bits enviado por el cliente sea compartido, en promedio, por al menos $k = 256$ hashes de la base (para que el servidor no pueda reducir el conjunto de sospechosos a un grupo menor a 256 emails). Si N es la cantidad de registros (emails) en la base y se asume que el hash distribuye uniformemente sus salidas entre los 2^x prefijos posibles (consecuencia del efecto avalancha del hash), la cantidad esperada de hashes por cada prefijo de x bits es $N/2^x$. El criterio para garantizar 256-anonymity es entonces pedir que esa cantidad esperada sea al menos 256:

$$\frac{N}{2^x} \geq 256 \iff x \leq \log_2 \left(\frac{N}{256} \right).$$

Es decir, el valor x (bits de prefijo a enviar) debe fijarse como $\lceil x \leq \log_2(N/256) \rceil$, con N el tamaño de la base de datos: cuantos menos bits de prefijo se envíen, más grande es el grupo de hashes indistinguibles entre sí (y al revés, si x crece, el grupo de hashes que matchean se achica y se pierde anonimato). **Nota de fidelidad:** esta fórmula se deriva aplicando la definición de k -anonimato dada en el propio enunciado junto con las propiedades de hash de la materia (salida fija, efecto avalancha $\Rightarrow$ distribución aproximadamente uniforme de prefijos); el concepto de k -anonimato en sí no figura en los resúmenes de la materia, así que no se cita ningún resultado formal de la cátedra específico de k -anonimato.

3.2 Ejercicio 2 — Red de sensores de oleoducto: CHACHA20 + HMAC-SHA3

Consigna (textual)

Un oleoducto es una tubería que transporta petróleo a grandes distancias (a veces miles de kilómetros). Dada la longitud, la dificultad de acceso en segmentos soterrados y el costo de no poder utilizarlo, el diseño incluye un sistema de control activo que permite identificar problemas en el transporte con una localización precisa de dónde ocurre el problema.

Cada 200m se emplazan un conjunto de sensores dentro del oleoducto que monitorean la presión y algunas características químicas que permiten estimar problemas como una fuga o contaminación. Estos sensores están conectados físicamente con un cableado interno, y cada 10 km hay concentradores, que son dispositivos externos a la tubería, que están conectados tanto al segmento anterior como al posterior al dispositivo y recopilan la información. Los concentradores transmiten por radiofrecuencia los resultados a una central, donde son analizados para generar alarmas cuando sea necesario (por ejemplo, si falta información de algún sensor).

Dado el valor del material transportado, un escenario de ataque contemplado incluye a un grupo con acceso a tecnología sofisticada, capaz de tomar control físicamente de un concentrador y manipularlo, con el objetivo de “pinchar” un tubo y extraer petróleo en pequeñas cantidades (algunos camiones por día) para comercializarlo por canales no oficiales. Cada grupo de sensores tiene un identificador único público de 128 bits, un identificador único privado de 128 bits (que puede ser utilizado, por ejemplo, como clave, pero nunca es transmitido). Los sensores siguen un modelo de transmisión simple estilo token-ring para hacer uso del cable que los conecta, y cuando es su turno, transmiten un par (identificador, enc(valor sensado)), utilizando CHACHA20 como función de cifrado de flujo.

Los concentradores cuentan con su propio par de identificadores, pero solo la central conoce los identificadores privados del resto. Cada concentrador captura los mensajes transmitidos por cada sensor durante 1 minuto, tanto de la sección anterior como la posterior al dispositivo, y las envía en un único mensaje de la forma (identificador, msg1|msg2|...|msgn, mac(msg1|msg2|...|msgn)), utilizando para el mac su identificador privado y la función hmac-sha3.

Nota: a|b significa a seguido de b.

- Explicar qué debe hacer la central para verificar la integridad de los valores de cada sensor que recibió, antes de determinar si sus valores son normales o no. (1 pto)
- ¿Puede alguien con capacidad de reemplazar un concentrador (incluso averiguando su identificador privado) falsificar valores de sensores que la central de como validos? Explicar por qué no o como puede hacerlo. (1 pto)
- ¿Puede alguien con capacidad de reemplazar dos concentradores falsificar valores de sensores que la central de como validos? Explicar por qué no o como puede hacerlo. (1 pto)

Temas. Primera Parte, Clase 2 — Cifrado (§Cifrado de flujo; §Generador pseudoaleatorio; §no reutilizar la clave/keystream; §los cifrados de flujo no son seguros bajo múltiples cifrados con la misma clave). Primera Parte, Clase 3 — MACs y cifrado autenticado (§Qué es un MAC; §HMAC: MAC a partir de una función de hash, incluye explícitamente HMAC-SHA3; §infalsificabilidad / Mac-forge). Segunda Parte, Clase 8 — Principios de diseño (§mínimo privilegio / confinamiento, para el razonamiento del inciso c).

Qué necesitás saber.

- **MAC (Message Authentication Code):** terna $(\text{Gen}, \text{Mac}, \text{Vrfy})$ con clave secreta compartida; $\text{Vrfy}_k(m, t) = 1$ si y solo si t es una etiqueta válida de m bajo k . Sin la clave, no se puede generar ni verificar un tag válido (infalsificabilidad).
- **HMAC:** $\text{HMAC}_k(m) = H((k \oplus \text{opad}) \parallel H((k \oplus \text{ipad}) \parallel m))$; si H es libre de colisiones, HMAC es infalsificable. La materia menciona explícitamente HMAC-SHA1, HMAC-MD5 (evitar) y **HMAC-SHA3**.
- **Cifrado de flujo (CHACHA20 es una instancia de esta familia, aunque el nombre concreto “ChaCha20” no aparece nombrado en los resúmenes de la materia):** $e_k(m) = G(k) \oplus m$, con G un generador pseudoaleatorio. Es **determinístico**: la misma clave/semilla produce siempre el mismo keystream.
- **Los cifrados de flujo NO son seguros bajo múltiples cifrados con la misma clave** (como el OTP reusado): reusar la clave permite XOR-ear dos cifrados y cancelar el keystream, exponiendo relaciones entre los textos planos.

Notas y conexiones. El identificador privado del sensor actúa como **clave simétrica de cifrado de flujo** para los datos, y el identificador privado del concentrador actúa como **clave del HMAC** sobre el mensaje agregado. Son dos primitivas independientes con dos claves independientes (una por sensor, para confidencialidad; una por concentrador, para integridad/autenticación del agregado), lo que limita el **alcance del compromiso** de cada clave — la misma idea de compartimentar privilegios que aparece en la unidad de Principios de diseño (mínimo privilegio, confinamiento): comprometer una clave no debería dar poder sobre datos protegidos por otra clave distinta.

Resolución. a) Para verificar la integridad del mensaje agregado recibido de un concentrador, la central debe:

1. Identificar el concentrador emisor por su **identificador público** (que sí viaja en el mensaje).
2. Buscar el **identificador privado** de ese concentrador, que la central conoce (según el enunciado, “solo la central conoce los identificadores privados del resto”).
3. Recalcular ella misma $\text{HMAC}_{k_{\text{priv}}}(\text{msg}_1 \parallel \dots \parallel \text{msg}_n)$ sobre la concatenación de mensajes recibida, usando el identificador privado del concentrador como clave y la función `hmac-sha3` (misma construcción que usó el concentrador para generar el tag).
4. Comparar el valor recalculado con el `mac(.)` recibido en el mensaje: si coinciden, el mensaje **no fue alterado** en tránsito (íntegro y autenticado como proveniente de ese concentrador); si no coinciden, se descarta/marca como no confiable.

Solo después de esa verificación (recalcular y comparar el HMAC) tiene sentido que la central analice si los valores sensados son “normales o no”; verificar antes evita procesar datos manipulados como si fueran legítimos.

b) **Sí puede falsificar valores**, precisamente porque conoce el identificador privado del concentrador reemplazado: ese identificador privado es exactamente la **clave del HMAC** que la central usa para validar la integridad del mensaje agregado de ese concentrador. Un atacante que reemplaza físicamente el concentrador puede:

1. Inventar o alterar los valores sensados de esa sección ($\text{msg}'_1, \dots, \text{msg}'_n$, p.ej. para esconder la caída de presión que delataría el “pinchazo”).
2. Calcular él mismo $\text{HMAC}_{k_{\text{priv}}}(\text{msg}'_1 \parallel \dots \parallel \text{msg}'_n)$ con el identificador privado que conoce (o ya lo tiene disponible por haber comprometido físicamente el dispositivo, según el enunciado).
3. Enviar (identificador público, mensajes falsificados, HMAC recalculado con la clave correcta) a la central, que al recalculer el HMAC obtiene una coincidencia y da los valores por **válidos**.

Es decir, el HMAC protege contra un atacante que **no** conoce la clave (garantiza infalsificabilidad frente a un adversario sin oráculo/clave), pero no protege si el propio concentrador — y por lo tanto quien lo controla — es el poseedor legítimo de esa clave: quien tiene la clave puede generar tags válidos para cualquier mensaje que elija, exactamente como el emisor legítimo.

c) Comprometer **dos** concentradores le da al atacante control sobre dos identificadores privados (dos claves de HMAC) en vez de una, pero cada concentrador usa **su propio par de identificadores**, independiente del resto (según el enunciado, “los concentradores cuentan con su propio par de identificadores”). Por lo tanto:

- El atacante puede falsificar, de la misma manera que en b), los valores de **los dos segmentos** cubiertos por esos dos concentradores comprometidos (el doble de alcance que en b), pero nada más).
- **No puede** falsificar valores de sensores/concentradores fuera de esas dos secciones: no conoce los identificadores privados del resto de los concentradores ni de los sensores de otras secciones, y sin esa clave no puede generar un HMAC que la central acepte como válido para esos otros segmentos (por la infalsificabilidad del HMAC bajo una clave que el atacante desconoce).

En síntesis: comprometer m concentradores solo compromete la integridad de esos m segmentos, nunca la del resto de la red — consecuencia directa de que cada concentrador (y cada sensor) tiene su propia clave independiente, limitando el “radio de explosión” de cada compromiso individual (mismo principio de aislar privilegios/claves visto en la unidad de Principios de diseño).

3.3 Ejercicio 3 — Hipótesis de falla para pentest de app móvil de exchange

Consigna (textual)

Una exchange de criptomonedas está expandiendo su aplicación web actual con una nueva aplicación móvil nativa para Android y otra para iOS.

Se sabe que:

- La aplicación se comunica con el servidor mediante un API REST.
- La aplicación requiere que el usuario se registre con un email y una contraseña.
- Sin autenticarse en la aplicación, se puede ver un resumen del valor actual de compra y venta de cada criptomoneda listada, y datos de volumen de operaciones.
- Para ingresar dinero se puede utilizar una tarjeta de crédito o una cuenta bancaria. Es necesario registrar los datos del medio a utilizar. Para retirar dinero, solo se puede utilizar una transferencia bancaria.

- Los ingresos y salidas de dinero están gestionados por un sistema de pagos externo, AlphaPay, que es quien procesa las operaciones. El proveedor disponibiliza un api para llevar a cabo las transacciones. Dicha API procesa las transacciones, pero no almacena información de medios de pago, que sí almacena la plataforma para no requerir al usuario ingresar todos los datos de cuenta en cada transacción.
- La aplicación notifica por email cuando una orden de compra o venta se ejecuta, incluyendo el resumen de la misma.

Ante un pedido de realizar un pentest que abarque la app, establecer 4 hipótesis de falla **pertinentes** para el caso, basadas en problemas de seguridad en aplicaciones. Explicar **para cada una**, como probaría según la metodología (la acción concreta que ejecutaría para verificar o refutar la hipótesis). (4 puntos, 1 pto cada uno)

Aclaración: No utilizar problemas genéricos como hipótesis de falla. Plantear situaciones y lugares concretos donde podría estar el problema. Por ejemplo, en lugar de decir “puede haber un buffer overflow”, decir “es probable que haya un buffer overflow en la lectura del parámetro x que debe interpretarse como número y puede tener un buffer pequeño asumiendo que el número tiene menos de los dígitos de un long”.

Temas. Segunda Parte, Clase 6 — Pentesting (§Metodología de Hipótesis de Falla, pasos y ejemplos de hipótesis concretas: seguridad en APIs, relaciones de confianza). Segunda Parte, Clase 3 — Principios de diseño y vulnerabilidades (§mediación completa; §límite de confianza / trust boundary; §CWE Top 25: injection, buffer overflow). Segunda Parte, Clase 2 — Autenticación (§fuerza bruta / throttling). Segunda Parte, Clase 7 — Protección de datos (§PCI-DSS: acotar datos críticos, tokenización).

Qué necesitas saber.

- La **metodología de hipótesis de falla** exige, para cada hipótesis, describir cómo se **probaría** (verificaría o refutaría): analizando documentación/comportamiento, o — si es necesario — intentando explotar la vulnerabilidad de forma controlada y repetible/concluyente.
- **Mediación completa / trust boundary:** todo dato que cruza un límite de confianza (ej. del cliente — app móvil, que el atacante controla totalmente — al servidor) debe validarse en el lado de *mayor* confianza; no se puede confiar en valores calculados o enviados por el cliente.
- **PCI-DSS (protección de datos):** acotar la cantidad de datos críticos almacenados según su criticidad (no proteger todo por igual), y usar **tokenización** para que, si se filtran los datos guardados por la plataforma, no tengan valor por sí solos.
- **Fuerza bruta / throttling:** sin límite de intentos o política de contraseñas, un endpoint de autenticación es vulnerable a ataques de fuerza bruta/diccionario contra cuentas existentes.

Notas y conexiones. El enunciado da varias pistas textuales que apuntan directamente a puntos débiles ya vistos en la materia: (1) la plataforma **almacena datos de medios de pago** (tarjeta/cuenta bancaria) para no pedirlos en cada transacción — esto es justo lo que PCI-DSS busca acotar/tokenizar, y es un dato crítico guardado localmente en vez de delegado al proveedor externo (AlphaPay), que explícitamente *no* los almacena; (2) el flujo de compra/venta involucra un valor de cotización y un monto que el cliente podría manipular si el backend no revalida contra el servidor/AlphaPay (mediación completa); (3) el registro con email/contraseña es un endpoint de autenticación clásico para fuerza bruta si no hay throttling; (4) los emails de notificación de operaciones incluyen un “resumen” cuyo contenido podría filtrar de más.

Resolución. Hipótesis 1 — Almacenamiento inseguro de datos de medios de pago (PCI-DSS). Es probable que los datos del medio de pago (número de tarjeta de crédito o datos de la cuenta bancaria) que la plataforma guarda para no pedirlos en cada transacción se almacenen en la base de datos de la aplicación sin la protección exigida por PCI-DSS (sin tokenización, es decir, guardando el dato real en vez de un token que apunte a él en un sistema aparte ultra-protegido) y/o asociados incorrectamente al usuario, permitiendo que un medio de pago registrado por un usuario sea reutilizable desde otra cuenta. *Prueba:* con dos cuentas de prueba, registrar un medio de pago en la cuenta A y luego, desde la cuenta B, intentar usar el identificador interno de ese medio de pago (obtenido por ejemplo interceptando el tráfico de la API REST de la cuenta A) en una operación de ingreso de dinero; verificar si la transacción se acredita/ejecuta contra el medio de pago de otro usuario. Adicionalmente, revisar (con acceso de caja gris/blanca) si el campo del medio de pago se guarda en texto plano o tokenizado en la base.

Hipótesis 2 — Confianza indebida en valores enviados por el cliente al ejecutar una orden. Es probable que, al confirmar una compra o venta de una criptomoneda, el backend confíe en el monto y/o el precio de cotización enviados desde la app móvil por el API REST al iniciar la orden, en lugar de revalidar esos valores contra la cotización real del servidor en el momento de ejecutar la operación — el cliente móvil cruza el límite de confianza (trust boundary) hacia el servidor y ese es el punto donde debería mediar completamente la validación. *Prueba:* interceptar (con un proxy) la request que envía la app al confirmar una orden de compra/venta, modificar el parámetro de precio o monto antes de que llegue al servidor, y verificar si la orden se ejecuta con el valor alterado en vez de con la cotización real vigente en el momento de la ejecución.

Hipótesis 3 — Falta de límite de intentos (rate limiting) en el endpoint de registro/login con email y contraseña. Es probable que el endpoint de autenticación (registro/login con email y contraseña) no implemente throttling ni bloqueo de cuenta tras varios intentos fallidos, permitiendo un ataque de fuerza bruta o de diccionario contra contraseñas de cuentas existentes. *Prueba:* realizar múltiples intentos de autenticación consecutivos contra una cuenta de prueba válida usando contraseñas incorrectas (diccionario de contraseñas comunes) y observar si el sistema bloquea, limita (con backoff) o registra/alerta sobre los intentos, o si permite continuar indefinidamente.

Hipótesis 4 — Exceso de información sensible en el email de notificación de operaciones. Es probable que el email que notifica la ejecución de una orden de compra o venta, al incluir “el resumen de la misma”, exponga información sensible en exceso (por ejemplo datos parciales del medio de pago usado, montos exactos de fondos disponibles, u otros identificadores internos de la cuenta/orden), quedando dicha información expuesta en un canal (el email) menos controlado que el API REST de la aplicación. *Prueba:* ejecutar una orden de compra/venta de prueba y analizar el contenido completo del email recibido, verificando si incluye datos sensibles (número parcial o completo de tarjeta/cuenta, identificadores internos, saldo total) que excedan lo estrictamente necesario para informar la operación.

Nota de fidelidad: las cuatro hipótesis se construyen a partir de datos concretos del propio enunciado (qué se registra, qué almacena la plataforma, qué notifica) combinados con los conceptos de la materia citados en “Qué necesitás saber” (PCI-DSS/tokenización, mediación completa/-trust boundary, throttling). No se incluyen hipótesis genéricas de OWASP Top Ten que no estén ancladas a un dato concreto del enunciado, siguiendo la aclaración del examen.

4 Final — 10/07/2023

4.1 Ejercicio 1 — Sensor → maestro con AES-CBC: integridad y replay

Consigna (textual)

Una empresa tiene un sistema donde hay varios sensores que comunican a un dispositivo maestro sus diferentes niveles de sensado. Intercambian claves con Diffie-Hellman en su primera comunicación, y a partir de eso en todos los mensajes que los sensores envían (encriptados con AES-CBC) tienen la siguiente estructura:

$$\#sensor \parallel e_k(\#sensor \parallel valor_sensado \parallel datos_adicionales)$$

1. Hay un atacante que hace de alguna forma que los mensajes se alteren y tengan datos adicionales no válidos, generando que el dispositivo maestro tire una señal de alerta. ¿Cómo es posible? Encontrar una manera de solucionar este error en el protocolo sin nuevas claves. (1,5 puntos)
2. Hay otro atacante que está enviando repetidamente la señal de sensado de un sensor, haciendo que el dispositivo maestro registre niveles de sensado que no son correctos. ¿Cómo es posible? Encontrar una forma de modificar el mensaje para que esto no ocurra sin usar nuevas claves. (1,5 puntos)

Temas. Primera Parte, Clase de MACs y Cifrado Autenticado (§El nuevo escenario de ataque: integridad; §Maleabilidad; §Cifrado autenticado: Encrypt-then-MAC); Clase de Protocolos (§Nonce y frescura; §Replay; §Diffie-Hellman como intercambio de clave).

Qué necesitás saber.

- **El cifrado por sí solo no da integridad.** El material lo remarca con el ejemplo del sistema de RR.HH.: un criptosistema CPA-Secure (incluso uno “bueno”) no impide que un atacante *manipule* el texto cifrado sin conocer la clave. La cita del profesor: “*En general el problema de integridad no es evitar la modificación, porque muchas veces no se puede, sino detectarla.*”
— dicho en clase
- **Maleabilidad:** propiedad de que modificar el texto cifrado se traduce de forma sistemática/controlable en una modificación del texto plano. En un cifrado de flujo, $c' = c \oplus x \Rightarrow \text{Dec}_k(c') = m \oplus x$. AES-CBC también es maleable: modificar un bloque de cifrado altera de forma predecible (vía el XOR con el bloque anterior) el bloque de texto plano correspondiente, y además **corrompe** el bloque siguiente al descifrar — pero eso no evita que el atacante logre inyectar datos con sentido si conoce o controla estructura del mensaje.
- **Ningún criptosistema visto es CCA-Secure** (recuadro “destacado” del material): el cifrado está pensado para confidencialidad, no para resistir manipulación. Por eso la integridad requiere **otra primitiva: el MAC**.
- **Solución sin cambiar claves — Cifrado autenticado / MAC:** el material define la construcción **Encrypt-then-MAC** como la única con demostración general de seguridad: se cifra m obteniendo c , y se etiqueta con $t = \text{Mac}_{k_2}(c)$ con clave **independiente** $k_2 \neq k_1$; el receptor **primero verifica** el tag y solo si es válido descifra. Alternativamente (y sin agregar una clave nueva, ya que acá solo se cuenta con k derivada de DH) se puede usar un **HMAC con una clave derivada de la misma k** (p. ej. vía un KDF/hash, tal como

aparece en el ejercicio análogo del material: $c = E_{k_1}(m \| H(k_2 \| m))$, donde k_1, k_2 salen de la misma clave maestra) para no necesitar un nuevo intercambio DH.

- **Nonce, timestamp y replay (Clase de Protocolos):** un **nonce** (number used once) es un número aleatorio de un solo uso que aporta frescura y permite detectar/evitar **replays** (repetición de mensajes viejos); el otro mecanismo de frescura es el **timestamp**. El material lo usa en Needham-Schroeder y en la variante Denning-Sacco justamente para evitar el reenvío de mensajes/claves viejas.

Notas y conexiones. El enunciado describe un mensaje cifrado con AES-CBC pero **sin ningún mecanismo de integridad ni de frescura**: no hay MAC sobre el mensaje, y no hay nonce, contador ni timestamp dentro de lo cifrado. Esto es exactamente el escenario que la unidad de MACs identifica como el “nuevo escenario de ataque”: un cifrado CPA-Secure (o incluso CCA-inseguro, que es la realidad de todos los criptosistemas vistos) puede ser manipulado sin que el atacante conozca la clave. La restricción “sin nuevas claves” en ambos incisos apunta a agregar mecanismos (MAC derivado de la misma clave, contador) en vez de rehacer el intercambio Diffie-Hellman.

Resolución. 1) **Por qué es posible y cómo solucionarlo.** AES-CBC cifra pero no **auténtica**: es un modo de cifrado por bloques que da confidencialidad (si el criptosistema es CPA-Secure), pero — como remarca el material — **ningún** criptosistema visto es CCA-Secure, es decir, ninguno resiste manipulación del texto cifrado. Un atacante que intercepta el mensaje puede alterar directamente el texto cifrado $e_k(\dots)$ (por ejemplo, invirtiendo bits de algún bloque cifrado, lo cual — por la estructura de CBC — modifica de forma predecible/controlable el bloque de texto plano correspondiente al descifrar, aunque corrompa el bloque siguiente). Como no hay ningún chequeo de integridad, el dispositivo maestro descifra “basura” que cae en el campo **datos_adicionales** sin que el receptor pueda detectar que fue modificado — eso es lo que dispara la alerta (datos adicionales no válidos), sin que el atacante haya necesitado la clave.

Solución (sin nuevas claves): agregar integridad con un **MAC/HMAC** calculado sobre el mensaje (siguiendo el patrón Encrypt-then-MAC del material, que es el único con demostración de seguridad), usando una clave derivada de la misma k obtenida por Diffie-Hellman (no hace falta un nuevo intercambio):

$$c = e_k(\#sensor \| valor_sensado \| datos_adicionales) \| m, \quad m = \text{HMAC}_k(c)$$

El maestro, al recibir $(\#sensor, c, m)$, **primero verifica** m contra c recalculando el HMAC; si no coincide, descarta el mensaje directamente (nunca llega a descifrar datos manipulados). Solo si el MAC es válido, descifra y usa **datos_adicionales**. Así cualquier alteración del cifrado es detectada antes de generar una alerta espuria.

2) **Por qué es posible y cómo solucionarlo.** El segundo atacante no modifica el contenido: **reenvía (replay)** un mensaje legítimo capturado anteriormente. Como la estructura del mensaje no incluye ningún elemento de **frescura** (ni nonce, ni contador, ni timestamp), el dispositivo maestro no tiene forma de distinguir un mensaje nuevo de uno viejo reenviado, y aun con un MAC válido (agregado en el punto 1) el mensaje *reenviado* seguiría verificando correctamente — el MAC da integridad y autenticación de origen, pero no dice nada sobre si el mensaje es *nuevo*. Por eso el maestro termina registrando niveles de sensado viejos como si fueran actuales.

Solución (sin nuevas claves): agregar un **contador (o timestamp)** dentro del contenido protegido por el MAC, tal como el material propone con la variante Denning-Sacco (agregar un timestamp T dentro del bloque protegido para impedir reutilizar un mensaje/ticket viejo):

$$c = e_k(\#sensor \| valor_sensado \| datos_adicionales \| n_i) \| m, \quad m = \text{HMAC}_k(c)$$

donde n_i es un contador estrictamente creciente por sensor (o un timestamp). El maestro guarda el último contador válido recibido de cada sensor y **rechaza** cualquier mensaje cuyo contador/-timestamp no sea mayor al último aceptado, aun cuando el MAC verifique correctamente. Esto

detecta y descarta el replay sin necesidad de repetir el intercambio Diffie-Hellman ni generar nuevas claves.

4.2 Ejercicio 2 — Alternativas de MAC para datos grandes (árbol de Merkle)

Consigna (textual)

Se quiere enviar un conjunto de datos muy grande. Si bien se puede mandar la información junto con un MAC de la información entera, en caso de que una parte del mensaje tenga un error, es necesario mandar el mensaje entero de vuelta. Se buscan alternativas para MAC, donde se parte la data en bloques más pequeños, de forma que permitan no tener que enviar la información entera en caso de una falla en uno de los bloques. Las alternativas son las siguientes:

1. En vez de calcular el MAC de cada bloque, se calcula el MAC' que es: el contenido del bloque anterior concatenado al MAC del bloque actual. Excepto en el bloque cero, que como no tiene anterior, se ponen 0s. (1 punto)
2. Se calcula el MAC de la concatenación de los MAC de cada bloque. (1 punto)
3. Se calcula el MAC usando el árbol de Merkle (buscar ejemplo de árbol de Merkle). (1 punto)

Para cada caso evaluar si tiene integridad y en caso de que sí, cómo harían cuando un bloque se envía con un dato erróneo (si es que hay una alternativa a no enviar todo el mensaje de vuelta).

Temas. Primera Parte, Clase de MACs y Cifrado Autenticado (§Qué es un MAC; §Falsificación por bloques (parcial 2015); §CBC-MAC; §HMAC; §Modelo iterativo Merkle-Damgård).

Aviso de fidelidad — el árbol de Merkle NO está en el material de la materia

El enunciado dice explícitamente “buscar ejemplo de árbol de Merkle”: el **árbol de Merkle (hash tree)** como estructura de datos **no aparece** en los resúmenes de la materia (ni en la unidad de MACs ni en ninguna otra). Lo que sí está en el material es la construcción **Merkle-Damgård** (el esquema iterativo genérico con el que se construyen las funciones de hash como SHA-2, encadenando $H_i = f(H_{i-1}, x_i)$) — que es un concepto **distinto** (sirve para construir una función de hash sobre un mensaje secuencial, no es una estructura de árbol para autenticar bloques independientes). Por lo tanto, la resolución de la alternativa 3 usa la noción *general* de MAC (Clase de MACs: Mac_k sobre una función de hash / HMAC) aplicada de forma recursiva sobre pares de hojas, que es la idea estándar de un árbol de hashes, pero **esa estructura en árbol específica no proviene de las clases provistas**: se avisa siguiendo la regla de oro en vez de presentarla como si estuviera dictada.

Qué necesitás saber.

- **Qué exige un MAC (Clase de MACs):** terna $(\text{Gen}, \text{Mac}, \text{Vrfy})$; $\text{Vrfy}_k(m, t) = 1$ si t es la etiqueta correcta de m . Un MAC **robusto siempre procesa la totalidad** del contenido que dice proteger: si deja algo afuera, ese resto queda **sin proteger** y es falsificable modificándolo libremente.
- **Falsificación por bloques (parcial 2015, en el material):** cuando un MAC se define como bloques **calculados de forma independiente** ($\text{Mac}_k(m) = \langle F_k(m_1), F_k(F_k(m_2)) \rangle$)

y similares), el ataque típico es **reusar/intercambiar bloques** entre mensajes distintos para forjar un tag válido de otro mensaje — justamente *porque* cada bloque se verifica de forma independiente. Esto es una debilidad de *seguridad* (permite forjar combinando piezas de mensajes distintos con MACs válidos), pero la misma independencia es la que permite, del lado de la **detección de errores de transmisión** (no forjas activas, sino bloques corrompidos), **verificar cada bloque por separado** y así saber exactamente cuál falló.

- **CBC-MAC (Clase de MACs):** $t_i = F_k(t_{i-1} \oplus m_i)$, el tag final “depende” de todos los bloques anteriores por el encadenamiento, pero solo se conserva/verifica el **último** tag t_j : no hay forma de saber, a partir de un único CBC-MAC roto, cuál bloque intermedio fue el que falló (hay que volver a mandar todo el mensaje para poder recalcular y comparar).

Notas y conexiones. El punto clave a verificar es la alternativa 2. En el material, la idea de que un MAC **calculado independientemente por bloque** permita identificar exactamente qué bloque fue alterado es consistente con la sección de “Falsificación por bloques”: ahí el material señala que esos esquemas son atacables *justamente porque* cada $F_k(m_i)$ se computa y se verifica de forma aislada del resto. Esa misma propiedad (verificación aislada por bloque) es la que, en el escenario de este ejercicio (corrupción/error de un bloque, no un atacante activo forjando), permite decir **cuál bloque en particular no verifica**. Por lo tanto, el razonamiento de que “si falla un bloque hay que reenviar todo el mensaje porque no se puede saber cuál bloque falló” para la alternativa 2 **no es correcto** y se corrige en la resolución de abajo.

Resolución. 1) $MAC'_i = \text{contenido del bloque anterior} \parallel \text{MAC del bloque actual}$ ($MAC'_i = B_{i-1} \parallel \text{Mac}(B_i)$, con $B_{-1} = \text{ceros}$).

- **¿Tiene integridad?** Sí, para el contenido de cada bloque: cada $\text{Mac}(B_i)$ se calcula (según la definición del material) sobre B_i y se puede verificar de forma independiente por bloque, exactamente igual que en la alternativa 2. Además, al incluir el contenido crudo de B_{i-1} junto al tag de B_i , el receptor puede **cotejar que el bloque anterior recibido coincida** con el que está referenciado en MAC'_i : esto agrega una verificación de **orden/posición** (detecta si se reordenaron o eliminaron bloques) que la alternativa 2 no ofrece por sí sola, a costa de duplicar el contenido de cada bloque (se transmite B_{i-1} dos veces).
- **Ante un bloque con error:** como $\text{Mac}(B_i)$ se calcula solo sobre B_i (no está encadenado criptográficamente con los bloques anteriores, a diferencia del CBC-MAC), el fallo se localiza en el bloque i cuyo $\text{Mac}(B_i)$ no verifica. Alcanza con **reenviar únicamente el bloque B_i** (el receptor ya tiene, del propio MAC'_{i+1} , la copia correcta del contenido de B_i para volver a verificar la cadena de orden una vez corregido).

Corrección respecto de una lectura errónea

No corresponde afirmar que, si falla un bloque, “hay que reenviar desde ese bloque y el siguiente”: dado que $\text{Mac}(B_i)$ en esta construcción se calcula únicamente sobre B_i (no sobre B_i encadenado con B_{i-1} dentro de la función de MAC), la localización del error es exacta en el bloque i , y MAC'_{i+1} no cambia al corregir B_i (ya traía la copia correcta desde el envío original).

2) $MAC = \text{MAC}(B_1) \parallel \dots \parallel \text{MAC}(B_n)$.

- **¿Tiene integridad?** Sí: si se altera cualquier bloque B_i , su $\text{Mac}(B_i)$ correspondiente deja de verificar (asumiendo un MAC infalsificable como los del material: CBC-MAC o HMAC), mientras que los tags de los demás bloques siguen siendo válidos.

- **Ante un bloque con error — corrección del razonamiento del alumno:** sí es posible saber exactamente cuál bloque falló, y por lo tanto **no hace falta reenviar el mensaje entero**. Como cada $\text{Mac}(B_i)$ se computa y se verifica de forma **independiente** (el mismo punto que el material remarca en “Falsificación por bloques”: cada bloque se calcula por separado), el receptor recalcula $\text{Mac}(B_i)$ para cada i y lo compara contra la posición i de la cadena recibida; el (o los) bloque(s) cuyo MAC no coincide son exactamente los que llegaron corruptos. Alcanza con **reenviar solo ese/esos bloque(s)**.
- Este es justo el punto que había que revisar contra el material: la independencia de los MAC por bloque, que en “Falsificación por bloques” se señala como una **debilidad de seguridad** (permite reusar/intercambiar bloques entre mensajes distintos para forjar un tag), es la **misma propiedad** que — en este contexto de detección de errores de transmisión, sin un atacante activo forjando — permite **localizar con precisión** el bloque dañado. No hay contradicción: son dos consecuencias de la misma independencia, una negativa (falsificabilidad por recombinación) y una positiva (localización de errores) en este ejercicio.

3) MAC calculado con un árbol de Merkle. Como se indicó en el aviso de fidelidad de arriba, la estructura específica del árbol de Merkle no está desarrollada en el material de la materia (solo Merkle-Damgård, que es otra cosa); se resuelve con la idea general de hash/MAC recursivo sobre pares de bloques, avisando que el detalle de la estructura en árbol excede las clases provistas:

- Cada hoja se etiqueta con el hash/MAC de su bloque: $h_i = \text{Mac}(B_i)$ (o $H(B_i)$ si se usa una función de hash del material, p. ej. SHA-3). Cada nodo interno combina sus dos hijos: $h = \text{Mac}_k(h_{\text{izq}} \| h_{\text{der}})$, hasta llegar a una única **raíz** que resume todo el conjunto de datos.
- **¿Tiene integridad?** Sí: si se modifica cualquier bloque hoja, su hash cambia, y ese cambio se propaga hacia arriba modificando todos los hashes de los nodos ancestros hasta la raíz — cualquier alteración es detectable comparando la raíz recalculada contra la raíz original.
- **Ante un bloque con error: no hace falta reenviar todo el mensaje.** Alcanza con reenviar el bloque hoja corregido junto con los hashes “hermanos” del camino desde esa hoja hasta la raíz (un camino de altura $\log_2 n$), de forma que el receptor pueda recalcular únicamente esa rama y verificar que la nueva raíz coincide con la original. Esto es más eficiente en ancho de banda de verificación que la alternativa 2 cuando n es muy grande (overhead logarítmico en vez de lineal en la cantidad de bloques), aunque ambas permiten localizar y corregir un único bloque sin reenviar el mensaje completo.

Resumen comparativo: las tres alternativas dan integridad y, contrariamente a enviar un único MAC sobre el mensaje entero (que obliga a reenviar todo ante cualquier error), las tres permiten identificar el bloque afectado y reenviar solo lo necesario: la 1 y la 2 con overhead lineal (un tag por bloque, más en la 1 el contenido duplicado del bloque anterior para chequeo de orden), y la 3 (árbol de Merkle, fuera del material provisto pero pedida explícitamente por el enunciado) con overhead logarítmico por verificación.

4.3 Ejercicio 3 — Vulnerabilidades de un sitio de noticias

Consigna (textual)

Se tiene una página de noticias donde hay usuarios que pueden ratear una noticia positiva o negativamente. Los publicadores de noticias pueden subir noticias pegándola a una API REST. Los usuarios interactúan con la página a través de una página web que se comunica con la API. Los ingresos de las publicidades de las noticias se pagan a los creadores según la cantidad de visitas que tiene una noticia.

Se encontraron 4 vulnerabilidades:

1. Para evaluar la positividad de una noticia, los usuarios tocan un botón que hace un GET a `/news_id/positive` o `/news_id/positive/negative`, pero se encontró que se puede hacer SSRF a través de esto.
 2. El contenido de una noticia se postea tal cual como se recibe, sin hacer validaciones.
 3. Las cookies de sesión no son `http_only`, y se pueden acceder a través de javascript.
 4. Un publicador de noticia hace un llamado al backend con una API KEY que se introduce en el pedido de la siguiente forma: `/api/{API_KEY}/publish`.
- a) Explicar cómo podría un atacante sumarse dinero de forma maliciosa con los puntos 2 y 3. (1,5 puntos)
 - b) Explicar cómo tomaría ventaja un atacante de la vulnerabilidad 1. (1,5 puntos)
 - c) Explicar por qué es inseguro que se envíen las API KEYS como lo indica el punto 4 e indicar cómo lo corregirían. (1 punto)

Temas. Segunda Parte, Clase de Principios de Diseño y Vulnerabilidades (§Vulnerabilidades web: XSS, CSRF, XXE; §Buffer overflow e inyección de código, como marco general de “mezclar datos con código/control”).

Aviso de fidelidad — varios términos del enunciado no están (con ese nombre) en el material

- **SSRF (Server-Side Request Forgery)** no aparece con ese nombre en los resúmenes de la materia. Lo más cercano en el material es **XXE (XML External Entities)**: “XML permite cargar esquemas desde una URL; si esa URL se usa para hacer un GET a otra página, un atacante puede escanear la red interna visible por el servidor”. Es el mismo patrón conceptual (el servidor hace, por instrucción del atacante, un request a un destino que no controla el usuario) pero el material lo desarrolla solo para el caso XXE, no como SSRF general vía un endpoint REST. Se marca para que quede claro que la resolución de b) se apoya en esa analogía y no en un contenido de SSRF dictado explícitamente.
- El atributo de cookie **http_only** no se menciona explícitamente en el material (sí se cubre, en la unidad de Vulnerabilidades web, que XSS permite “robar la sesión” ejecutando JS en el browser de la víctima, que es la consecuencia directa de no tener esa protección).
- Los mecanismos concretos de **transporte seguro de API keys** (headers, OAuth/JWT) y el problema de que un secreto en la URL quede en **logs, historial del navegador o se filtre por el header Referer** tampoco están desarrollados explícitamente en el material provisto. La resolución de c) se apoya en los principios generales de diseño del material (mediación completa, fallar de forma segura, mínimo privilegio) más el criterio general de no exponer secretos donde quedan registrados, pero se avisa que el material no cubre el detalle técnico de dónde específicamente queda expuesta una URL.

Qué necesitás saber.

- **XSS (CWE-79, Clase de Vulnerabilidades web):** inyección de JavaScript que termina ejecutado por el browser de la víctima, aprovechando la confianza entre el sitio y la

sesión del usuario; ocurre cuando se hace render de HTML mezclando un dato controlado por el usuario con código sin escapar. Consecuencia remarcada en el material: **permite robar la sesión**. Defensa: sanitizar/escapar toda salida (*escape on output*).

- **“El contenido se postea tal cual, sin validaciones”** (vulnerabilidad 2) es exactamente la falta de sanitización de entrada/salida que el material describe como causa de XSS: el dato del publicador (potencialmente conteniendo `<script>`) queda embebido en la página que ven los demás usuarios sin escapar — un **XSS almacenado** (*stored XSS*), ya que el payload queda guardado en la noticia y se ejecuta cada vez que otro usuario la visita.
- **XXE (Clase de Vulnerabilidades web)**: el servidor termina haciendo un request (GET) a una URL/destino determinado por un input controlado por el atacante, lo que permite usar al servidor como intermediario para alcanzar recursos que el atacante no podría alcanzar directamente (p. ej. la red interna).
- **Principios de Saltzer & Schroeder (Clase de Principios de Diseño)**: *mediación completa* (todo acceso debe pasar por un chequeo, sin atajos) y *fallar de forma segura* son relevantes para pensar por qué exponer un secreto en una URL es delicado: la URL viaja/queda registrada por canales que no están pensados para llevar secretos.

Notas y conexiones. Las vulnerabilidades 2 y 3 se combinan de forma clásica: la vulnerabilidad 2 (falta de sanitización) es la que **habilita** el XSS, y la vulnerabilidad 3 (cookies sin protección adicional contra acceso por JavaScript) es lo que hace que ese XSS pueda **robar la sesión** concretamente (tal como dice el material sobre XSS: “permite robar la sesión”). La vulnerabilidad 1 es un patrón análogo al XXE del material (el servidor hace un request a donde el atacante decide). La vulnerabilidad 4 conecta con los principios de diseño generales (no exponer secretos por canales inadecuados), aunque — como se avisó arriba — el material no desarrolla el detalle de headers/OAuth/JWT.

Resolución. a) **Puntos 2 y 3 — cómo un atacante se suma dinero de forma maliciosa.** Como el contenido de una noticia se postea **sin ninguna validación** (vulnerabilidad 2), un publicador (o cualquiera con acceso a publicar) puede subir una noticia cuyo “contenido” incluya un `<script>` malicioso: es un caso de **XSS almacenado** (el material define XSS como inyección de JS que se ejecuta en el browser de la víctima al renderizar HTML sin escapar un dato controlado por el atacante). Cuando otro usuario visita esa noticia, el JS inyectado se ejecuta en su browser, con la confianza de la sesión de ese usuario en el sitio.

Como además las cookies de sesión **no son http_only** (vulnerabilidad 3), ese JavaScript malicioso **puede leer la cookie de sesión** de cualquier usuario que visite la noticia (incluida la de otro publicador o incluso de un administrador) y enviarla al atacante — exactamente la consecuencia que el material atribuye a XSS: “permite robar la sesión”. Con la cookie de sesión robada, el atacante puede **hacerse pasar por ese usuario** (*masquerading*) y usar su identidad autenticada, por ejemplo, para hacer que el sistema le atribuya a sí mismo (o a una cuenta propia) visitas/ratings sobre sus propias noticias, o directamente actuar en el backend con los privilegios de publicador robado. Dado que el modelo de negocio paga según **cantidad de visitas**, el atacante puede automatizar, con las sesiones robadas, visitas/ratings positivos masivos y falsos sobre sus propias noticias publicadas, generando así ingresos por publicidad de forma fraudulenta sin que el pago esté respaldado por tráfico/interés real.

b) **Ventaja que un atacante saca de la vulnerabilidad 1 (SSRF).** El endpoint `/news_id/positive` (o `/news_id/positive/negative`) hace que el **servidor** realice una acción a partir de un parámetro (`news_id`) que el atacante controla. Si ese parámetro puede manipularse para que en lugar de identificar una noticia el servidor termine haciendo un request HTTP hacia una URL/host arbitrario elegido por el atacante, se está en el mismo patrón que el material describe para **XXE**:

el servidor actúa como intermediario y hace peticiones a destinos que el atacante no podría alcanzar directamente. Un atacante puede aprovechar esto para: hacer que el servidor (que sí tiene acceso) haga requests hacia **recursos internos de la red** que no son alcanzables desde afuera (por ejemplo, otros servicios internos, o un endpoint de otra noticia/recurso que solo el backend puede alcanzar), usando al servidor de noticias como “proxy” para escanear o alcanzar esa red interna — igual que en el ejemplo de XXE del material, donde el atacante “puede escanear la red interna visible por el servidor”. También puede usarlo para forzar al servidor a hacer pedidos repetidos con fines de denegación de servicio hacia un tercero, usando la infraestructura del sitio como origen del ataque.

c) **Por qué es inseguro poner la API KEY en la URL (/api/{API_KEY}/publish) y cómo corregirlo.** Es inseguro porque la API KEY, al viajar como parte de la **URL**, deja de ser un secreto protegido y queda expuesta por canales que no están pensados para llevar secretos: cualquiera que pueda ver esa URL (quien mire por encima del hombro, quien tenga acceso a un proxy intermedio, o quien pueda ver dónde queda registrada) accede directamente a la API KEY con la que se pueden publicar noticias en nombre de ese publicador. **Nota:** el material no detalla explícitamente todos los canales por los que una URL con secreto puede filtrarse (logs, historial del navegador, header **Referer**, cachés intermedias); esto excede lo cubierto en las clases provistas, aunque es consistente con el principio general de que un mecanismo de autenticación **no debería fallar de forma insegura** exponiendo la credencial por un canal que no la protege.

Además, tal como está planteado, no hay forma de saber si quien usa la API KEY es realmente el publicador dueño de esa clave: cualquiera que la obtenga puede usarla sin que el sistema pueda distinguir el uso legítimo del malicioso.

Corrección propuesta: no incluir el secreto en la URL; enviar la API KEY en un **header HTTP** dedicado a autenticación (p. ej. **Authorization**), que no queda expuesto por los mismos canales que la URL. Esto es consistente con la idea general de la materia de separar el canal de datos de control (autenticación) del canal de datos de la petición en sí, en vez de mezclarlos en el mismo campo que identifica el recurso.

5 Final — 2025, Segundo Cuatrimestre (1)

5.1 Ejercicio 1 — Pulseras Bluetooth (AES-CTR, modo degradado)

Consigna (textual)

Eran pulseras bluetooth para abrir puertas que cada dos segundos mandaban su (ID, CTR, C) donde CTR es un contador, $C = \text{AES-CTR}_k(CTR \parallel \text{bit_wearing})$. bit_wearing es un bit que indica si es o no una persona (si la pulsera está puesta).

El servidor, al recibir (ID, CTR, C) , descifra C con la K asignada a ese ID y compara si el CTR descifrado es el mismo que el que venía en claro; si lo es, le da el acceso.

Si el servidor se queda sin conexión, se entra en un **estado degradado** donde la puerta deja pasar a cualquier ID que haya tenido acceso recientemente, sin hacer ninguna otra validación.

- ¿Qué validaciones podría hacer el servidor para evitar comportamientos extraños como que se abran dos puertas distantes casi al mismo tiempo, o que se intenten abrir puertas fuera de un horario normal?
- Si alguien captura los mensajes, explicar cómo podría utilizarlos o modificarlos para tener acceso. ¿Cómo lo solucionaría?
- Si consiguen el estado degradado, ¿qué podría hacerse para que no puedan acceder repitiendo mensajes?

Temas. Primer Parte, Clase 2 — Cifrado (§Modo CTR: keystream $E_k(\text{nonce} \parallel i)$, nunca usa D_k , paralelizable, CPA-Secure solo si no se repite (k, nonce)); Primer Parte, Clase 3 — MACs y Cifrado Autenticado (§Maleabilidad de cifrados de flujo/CTR; §CCA y por qué el cifrado solo no alcanza para integridad; §MAC: infalsificabilidad, procesar todo el mensaje); Segunda Parte, Clase 2 — Autenticación (§OTP/HOTP: contador sincronizado, códigos de un solo uso, throttling); Segunda Parte, Clase 5 — Seguridad de la Empresa (§IDS/IPS: detección por anomalías; §SIEM: correlación de eventos y alertas).

Qué necesitás saber.

- **CTR es un cifrado de flujo, no autenticado.** $C_i = E_k(\text{nonce} \parallel i) \oplus M_i$; el descifrado es $M_i = E_k(\text{nonce} \parallel i) \oplus C_i$ (nunca se usa D_k). CTR da *confidencialidad* (si no se repite el par clave/nonce), pero **no** da integridad ni autenticación por sí solo.
- **Maleabilidad de cifrados de flujo (Clase 3).** Si $\text{Enc}_k(m) = G(k) \oplus m = c$, entonces $c' = c \oplus x \Rightarrow \text{Dec}_k(c') = m \oplus x$: un atacante que conoce (o adivina) un bit de plano en una posición conocida puede flippear ese bit del cifrado, y al descifrar cambia exactamente ese bit del plano, **sin conocer la clave**. Es el mismo mecanismo que el ataque al sueldo en RR.HH. del material.
- **Ningún criptosistema de los vistos es CCA-Secure:** el cifrado está pensado para confidencialidad, no para resistir manipulación del texto cifrado. Por eso la integridad requiere una primitiva aparte: el **MAC**. Un MAC robusto siempre procesa la *totalidad* del mensaje (si no, lo que queda afuera se puede alterar sin invalidar la etiqueta).
- **OTP/HOTP (Clase 2, Segunda Parte):** A y B comparten una clave K y un contador C ; el código depende de (K, C) y se recomienda *throttling* y (para HOTP) una ventana de *look-ahead* para tolerar desincronización del contador.

- **IDS/IPS y SIEM (Clase 5, Segunda Parte):** un IDS/IPS detecta por **anomalías** (además de firmas) y decide bloquear o alertar; un SIEM **correlaciona eventos** de distintas fuentes (aquí, aperturas de puertas distintas) para detectar patrones anómalos y generar alertas.

Notas y conexiones. El enunciado describe un esquema de *confidencialidad sin integridad*: se cifra ($CTR\|bit_wearing$) con AES-CTR, pero el mensaje completo (ID, CTR, C) no lleva ningún MAC que lo ate. Esto es exactamente el escenario que motiva la Clase 3 (“el cifrado por sí solo no protege contra modificaciones”): acá, además, el CTR viaja **en claro** junto al cifrado, lo que facilita un replay directo. La verificación descrita en el enunciado (“compara si el CTR es el mismo que el que venía en C ”) es débil porque no dice qué pasa si el CTR ya fue usado antes (no hay noción de *frescura/monotonidad*), y no ata ID al contenido cifrado con una clave de integridad.

Resolución. a) **Validaciones de contexto (correlación entre eventos, no solo por mensaje).** Estas validaciones no dependen de la criptografía del mensaje individual sino de **cruzar eventos en el servidor** (tipo SIEM: correlación de eventos + reglas de anomalía, IDS/IPS basado en anomalías):

- **Verificación de coherencia física (velocidad de desplazamiento):** el servidor conoce la posición de cada puerta. Si el mismo ID abrió una puerta y poco después (menos tiempo del físicamente necesario para recorrer la distancia) intenta abrir otra puerta lejana, la solicitud es **físicamente imposible** y debe rechazarse y generar una alerta (correlación de eventos por ID a través del tiempo y el espacio).
- **Ventanas horarias (ACL temporal):** asociar a cada ID (o a cada puerta) un rango horario habilitado en una base de datos; si llega una trama fuera de ese rango, denegar el acceso y generar una alerta de seguridad, en lugar de autorizar silenciosamente.
- En ambos casos el mecanismo es **detectivo/correlativo** (como un IDS/SIEM): no reemplaza la validación criptográfica del mensaje individual, la complementa detectando patrones anómalos entre mensajes.

b) **Captura de mensajes: uso/modificación y solución.** Como C se obtiene con AES-CTR (un cifrado de flujo), aplica directamente la **maleabilidad de los cifrados de flujo** vista en la Clase 3:

- **Replay puro:** el atacante captura un mensaje válido (ID, CTR, C) y lo reenvía más tarde. El enunciado solo dice que el servidor “compara si el CTR es el mismo que el que viene en C ” — *no* dice que el servidor lleve un registro del último CTR usado por ese ID . Si no lo lleva, el mensaje capturado **se vuelve a aceptar tal cual** (replay exitoso).
- **Modificación por maleabilidad (si el atacante conoce/adivina el bit de *wearing*):** dado que CTR es un XOR con el keystream ($C = E_k(\text{nonce}\|i) \oplus (CTR\|bit_wearing)$), sin conocer la clave k el atacante puede flippear el bit del cifrado que corresponde a la posición del bit $bit_wearing$ dentro de C : eso cambia exactamente ese bit al descifrar (igual que el ataque al sueldo: $c'' = c \oplus x$ cambia el bit correspondiente del plano), **sin necesitar la clave**. Así podría forzar a que el servidor interprete que la pulsera está puesta (o no) según convenga para pasar alguna validación adicional que dependa de ese bit.
- **Solución:** agregar **integridad** con un MAC sobre el mensaje completo ($ID\|CTR\|bit_wearing$), calculado con una clave de MAC (idealmente distinta de la de cifrado) y verificado antes de aceptar el mensaje. Un MAC que cubre la totalidad del contenido evita tanto la modificación por maleabilidad (cualquier bit flippeado invalida la etiqueta) como facilita atar

el ID al contenido cifrado. Además, el servidor debe llevar un **registro del último CTR aceptado por ID** y solo aceptar $CTR_{\text{nuevo}} > CTR_{\text{último}}$ (monotonidad / frescura), lo que bloquea el replay puro.

c) **Estado degradado: evitar replay sin conexión.** En el modo degradado el servidor no puede verificar nada dinámicamente (no tiene conexión para consultar estado), así que la defensa tiene que apoyarse en algo que **se consume** localmente, análogo a la idea de **OTP/HOTP** (Clase 2, Segunda Parte): claves de un solo uso derivadas de un contador compartido.

- Precargar en la memoria de la puerta (mientras hay conexión) una **lista de códigos de un solo uso** (equivalente a un HOTP: valores derivados de (K, C) con C incremental), vinculados a cada pulsera/ ID .
- Cuando no hay conexión, la pulsera envía el **siguiente código no usado** de esa lista y la puerta lo valida **localmente** y lo **marca como consumido** (borrándolo o avanzando un puntero), de forma que no pueda reutilizarse: esto es exactamente la garantía de un esquema de un solo uso (OTP), que rompe el replay porque cada código sirve una única vez.
- Al recuperar la conexión, el servidor debe re-sincronizar el estado (equivalente al *look-ahead* de HOTP) y reponer/renovar la lista de códigos consumidos localmente.

5.2 Ejercicio 2 — Protocolo de autenticación Cliente–Servidor ($H(K_c||r)$)

Consigna (textual)

Protocolo de autenticación entre un Cliente C y un Servidor S que ambos tienen una K_c compartida. La función H es un hash sin colisiones.

$$C \rightarrow S : ID$$

$$S \rightarrow C : r \quad (\text{número aleatorio})$$

$$C \rightarrow S : H(K_c||r)$$

El servidor calcula el hash usando la K_c que tiene guardada y el r . Si da lo mismo que recibió, autentica a C .

Preguntas:

1. ¿Por qué el servidor manda ese r ? ¿Qué pasaría si el cliente solo mandase $H(K_c)$ en vez de que se use el r ?
2. ¿Disminuiría la seguridad si se usara un contador incremental en vez del r aleatorio?
3. Plantear cómo se podría hacer para que el cliente autentique también al servidor, sin introducir nuevas claves.

Temas. Segunda Parte, Clase 2 — Autenticación (§Challenge-Response y EKE: el desafío r como nonce, “el secreto nunca viaja por la red”); Primer Parte, Clase 5 — Protocolos (§Ataque por reflexión y “romper la simetría” con nonces/identidad por sentido; §Handshakes con nonces de ambos lados, ej. TLS *Hello* con r_1, r_2).

Qué necesitas saber.

- **Challenge-response (Clase 2, Segunda Parte):** el servidor manda un **challenge único** r (nonce); el cliente responde $f(k, r)$ (acá, $H(K_c||r)$) probando que conoce el secreto K_c **sin enviarlo**. El diagrama del material remarca explícitamente: “el nonce evita reutilizar respuestas capturadas”.

- Si no hubiera r (o el cliente mandara siempre $H(K_c)$, un valor fijo), la respuesta sería **siempre la misma**: un atacante que la capture una vez puede **reenviarla** (replay) y autenticarse sin conocer K_c .
- Un **contador incremental** en vez de un r aleatorio también da frescura (cada valor se usa una sola vez si el servidor lo verifica creciente), pero es **predecible**: un atacante que observa la secuencia puede anticipar el próximo valor. En el material esto es exactamente el punto que distingue HOTP (contador, requiere sincronización, predecible en su progresión) de un valor **aleatorio** imprevisible.
- **Autenticación mutua sin nuevas claves**: el material resuelve el problema simétrico en los **handshakes** (Clase 5, Primer Parte) generando un desafío **en cada sentido** con la misma clave compartida, y remarca (a propósito del ataque por reflexión) que hay que **romper la simetría** entre ambos desafíos (por ejemplo, atando la identidad/dirección a cada desafío) para que una sesión paralela no pueda usarse para reflejar la respuesta.

Notas y conexiones. Este protocolo es el patrón **challenge-response** de la Clase 2 (Segunda Parte) aplicado con hash en lugar de una función $f(k, r)$ genérica. El ataque por **reflexión** de la Clase 5 (Primer Parte) es la advertencia clave para la pregunta 3: si simplemente se agrega un segundo desafío simétrico (el cliente manda un r_c y espera $H(K_c || r_c)$ del servidor, de la misma forma en que el servidor se lo pide al cliente), un atacante podría abrir una **sesión paralela** y reflejar el desafío recibido en una sesión para responder en la otra, si no se distingue el rol/dirección de cada desafío.

Resolución. 1) **Por qué se manda r ; qué pasa con $H(K_c)$ solo.** El servidor manda r para dar **frescura** a la respuesta: como r es distinto en cada intento de autenticación, la respuesta $H(K_c || r)$ también lo es, así que **capturar una respuesta pasada no sirve para autenticarse de nuevo** (no hay dos ejecuciones del protocolo con la misma respuesta válida). Si el cliente mandara directamente $H(K_c)$ (sin usar r), la respuesta sería **siempre el mismo valor fijo** en cada autenticación; un atacante que la capture una sola vez (escuchando el canal) puede luego **reenviarla (replay)** y el servidor la aceptaría como si fuera el cliente legítimo, sin conocer nunca K_c . Es exactamente el rol del nonce en challenge-response: “el secreto nunca viaja por la red” y **cada respuesta es única**, ligada a un desafío que no se repite.

2) **Contador incremental en vez de r aleatorio.** Sí **disminuye la seguridad** respecto de un nonce verdaderamente aleatorio, aunque igual aportaría algo de frescura si el servidor exige que sea creciente (evitaría el replay literal del mismo mensaje). El problema es la **predictibilidad**: si el contador es incremental y el atacante observa la secuencia (por ejemplo, escuchando varias rondas), puede **anticipar el valor del próximo contador** y, si además logra capturar u obtener de otra forma el valor de $H(K_c || r)$ para un r futuro conocido (o construir un ataque que aproveche esa previsibilidad, según qué tan expuesto esté el mecanismo), reduce el espacio de incertidumbre del protocolo. En el material esto es análogo a la comparación entre HOTP (contador, requiere sincronización explícita y su progresión es previsible) y un desafío aleatorio (TOTP/nonce, imprevisible salvo que se conozca la clave). Un valor **aleatorio e imprevisible** es preferible porque no permite a un atacante preparar de antemano una respuesta para un desafío futuro.

3) **Autenticación mutua sin introducir nuevas claves.** Usando la misma K_c compartida, se agrega un desafío **en el otro sentido**, rompiendo la simetría para evitar un ataque por reflexión (Clase 5, Primer Parte):

$$\begin{aligned} C &\rightarrow S : r_C, ID \\ S &\rightarrow C : r_S, H(K_c || r_C) \\ C &\rightarrow S : H(K_c || r_S) \end{aligned}$$

El servidor prueba que conoce K_c respondiendo al desafío r_C del cliente (en el mismo mensaje en que manda su propio desafío r_S); el cliente, a su vez, responde al desafío r_S . Cada parte verifica el hash correspondiente a *su propio* desafío. Es importante que ambos desafíos **no sean intercambiables uno por el otro** sin ambigüedad (en el material, la defensa contra la reflexión es distinguir la dirección/rol de cada nonce, por ejemplo incluyendo dentro del hash la identidad o el rol del que responde, del estilo $H(K_c \| r_S \| \text{"cliente"})$ y $H(K_c \| r_C \| \text{"servidor"})$) para que una sesión paralela abierta por un atacante no pueda usarse para que el servidor legítimo “firme” un desafío que luego se reutilice contra el cliente. No se introduce ninguna clave nueva: se reutiliza K_c y H en ambos sentidos.

5.3 Ejercicio 3 — CasaIA (IoT hogareño): hipótesis de falla

Consigna (textual)

Una página web y aplicación mobile conectadas con una API REST. Se llamaba CasaIA, te dejaba controlar los dispositivos de tu casa a través de wifi, y podrías configurar funciones por horario y por ubicación geográfica del teléfono. Los dispositivos están conectados con un websocket en TCP al servidor, que es el que les puede mandar comandos (abrir/cerrar, etc.). Los dispositivos se vinculan escaneando un QR que viene en ellos. Se puede dar acceso temporal a través de la página a otra persona: el usuario pone el mail de la persona y a esta le llega un mail con el cual, al abrirlo, consigue acceso temporal. También se genera un token de sesión al lograrse el acceso. Se pide: como siempre, dar **4 hipótesis de falla** y su prueba.

Temas. Segunda Parte, Clase 6 — Pentesting (§Metodología de hipótesis de falla, Paso 2: áreas candidatas de hipótesis; §la prueba debe ser repetible y concluyente); Segunda Parte, Clase 3 — Principios de Diseño y Vulnerabilidades (§CSRF; §XSS; §nonce/token anti-replay); Segunda Parte, Clase 2 — Autenticación (§OTP/HOTP, contador vs. aleatorio; §challenge-response); glosario (§OAuth2/tokens, §nonce).

Qué necesitás saber.

- **Metodología de hipótesis de falla (Clase 6):** el paso 2 consiste en plantear posibles vulnerabilidades **examinando implementaciones y mecanismos** (¿está bien implementado el control?, ¿el ambiente introduce errores?, ¿es realmente seguro?) y **comparando con sistemas parecidos** (sistemas similares tienen problemas similares). El paso 3 (prueba) exige que la verificación de la hipótesis sea **repetible y concluyente**.
- **Nonce / frescura (Clase 3):** la defensa real contra CSRF es un **token/nonce único** que dé frescura y evite replay; lo mismo aplica a cualquier enlace o token de un solo uso (como el mail de acceso temporal de CasaIA).
- **XSS (Clase 3):** inyectar código que termine ejecutado por el browser de la víctima permite **robar la sesión** (aprovechando la confianza del sitio en su sesión/cookie).
- **Tokens de sesión (glosario):** un token de sesión, si no tiene expiración adecuada, alcance limitado, o viaja sin protección, puede ser robado o reutilizado para suplantar al usuario.
- **OTP/HOTP (Clase 2):** un código de acceso temporal enviado por mail que actúa como un “one-time” debería, como en HOTP/TOTP, tener **throttling** y no ser reutilizable ni de vida indefinida.

Nota sobre alcance (regla de oro). El enunciado pide una **metodología puntual** (“4 hipótesis de falla, una prueba cada una”); en el material de Clase 6 (Pentesting) la metodología está descripta en **5 pasos** (Recolección → Hipótesis → Prueba → Generalización → Eliminación) y el paso de **Hipótesis** es explícitamente donde se listan posibles vulnerabilidades (no hay, en las clases provistas, una regla de puntaje del tipo “4 hipótesis, 1 punto cada una”; eso parece ser una convención de la consigna del parcial, no un contenido de la clase). Lo que sí está en el material es *cómo* se plantea y se prueba cada hipótesis (paso 2 y 3 de la metodología), que es lo que se aplica a continuación sobre CasaIA.

Resolución. Aplicando la metodología de hipótesis de falla (examinar implementación/mecanismos + comparar con sistemas parecidos) a los cuatro componentes distintivos de CasaIA (QR de vinculación, websocket de comandos, acceso temporal por mail, token de sesión):

1. **Hipótesis — vinculación por QR sin verificación de posesión.** Si el QR del dispositivo solo contiene un identificador estático (sin un secreto o desafío que se consuma una única vez), cualquiera que **fotografíe o copie** el QR (por ejemplo, publicado sin querer en una foto, o visto por un instalador) podría vincular ese dispositivo a su propia cuenta. *Prueba (repetible y concluyente, Clase 6):* tomar una foto del QR de un dispositivo ya vinculado y, desde otra cuenta, intentar vincularlo nuevamente; si el sistema lo permite (o no invalida la vinculación anterior), la hipótesis queda confirmada.
2. **Hipótesis — falta de autenticación/autorización en los mensajes del websocket.** Si el canal websocket entre servidor y dispositivo no reautentica cada comando (solo se autenticó la conexión TCP inicial) ni valida que el comando provenga de un usuario autorizado sobre *ese* dispositivo puntual, un atacante que logre interceptar o inyectar tráfico en ese canal podría mandar comandos (abrir/cerrar) sin ser el dueño. *Prueba:* con acceso a la red del dispositivo (caja gris), capturar el tráfico del websocket y reenviar (*replay*) un comando “abrir” capturado, o inyectar un comando propio, y verificar si el servidor lo ejecuta sin re-validar autorización.
3. **Hipótesis — el enlace de acceso temporal por mail no tiene frescura/expiración adecuada (análogo a CSRF/replay, Clase 3).** Si el link que llega por mail para dar acceso temporal es un token que no expira, no es de un solo uso, o es predecible/corto, cualquiera que lo intercepte (compartido reenvío de mail, mail comprometido, o el link cacheado en el navegador) podría usarlo para obtener acceso indefinidamente, igual que un token anti-CSRF débil no da frescura real. *Prueba:* generar un acceso temporal, usarlo una vez, y volver a usar el mismo enlace pasado un tiempo prudencial (o tras revocar el acceso desde la app); si sigue funcionando, la hipótesis queda confirmada.
4. **Hipótesis — robo/reutilización del token de sesión (estilo XSS/robo de sesión, Clase 3).** Si el token de sesión generado al lograrse el acceso (temporal o del dueño) no tiene alcance ni expiración acotados, o si la interfaz web es vulnerable a inyección de código (XSS) que permita leer ese token, un atacante podría robarlo y **suplantar la sesión** sin conocer ninguna contraseña. *Prueba:* intentar inyectar un script en algún campo de la aplicación web (por ejemplo, el nombre del dispositivo o de la persona invitada) y verificar si se ejecuta al ser renderizado por otro usuario (confirmando que el token/cookie de sesión de ese usuario quedaría expuesto); alternativamente, verificar si el token de sesión sigue siendo válido después de que expiró el acceso temporal que lo originó.

6 Ejercicios adicionales

6.1 Ejercicio — Bingo virtual

Consigna (textual)

Una franquicia de bingos decide crear una aplicación que simule un bingo virtual donde los participantes paguen y obtengan dinero real.

Cada juego tiene un valor de ingreso, y le permite al participante obtener un cartón con 15 números elegidos al azar entre 1 y 90. Cuando comienza el juego, el servidor comienza a “sacar” números, y los participantes deben ir marcando los números en su cartón si corresponde.

Cuando uno de los participantes completa los 15 números de su cartón, selecciona el botón “bingo”, y si realmente salieron todos, gana el premio de esa jugada y termina la partida para todos. Si no fuese así, se le avisa y se lo descalifica.

Resuelta la seguridad de los aspectos de introducir y retirar dinero se identificaron tres funcionalidades necesarias respecto a la dinámica del juego. Explicar como se podría lograr cada una recurriendo a funciones criptográficas simétricas solamente, justificando la elección. De ser necesario, explicar que información se intercambia en cada momento entre las apps y el servidor.

- El servidor debe generar una secuencia de números entre 1 y 90, que luego pueda ser reproducida por los clientes al finalizar la partida (para garantizar que no se favoreció adrede a alguien en el sorteo). (1 pto.)
- Los cartones se crean en las apps cliente al inicio de la partida. El servidor no conoce los cartones hasta el momento de verificar un bingo, pero debe ser posible para el servidor verificar que el cartón no fue modificado una vez empezado el juego (para evitar que un jugador arme el cartón ideal para ganar luego de 15 números). (1 pto.)
- La app de cada participante deberá poder verificar si el ganador realmente completó el bingo y emitir una alerta en caso contrario (para evitar que el servidor de por ganador a alguien que no completo el cartón). Las apps no se comunican entre si, sino a través del servidor. (1 pto.)

Nota: Considerar que hay aproximadamente 2^{55} cartones (combinatoria de 90 tomados de a 15), y asumir que un atacante podría llegar a probar esa cantidad de combinaciones.

Temas. Clase 3 — MACs y Cifrado Autenticado (§Qué es un MAC; §HMAC; §funciones de hash y sus propiedades de resistencia; §usos de la resistencia a preimagen para protocolos de compromiso/prueba de conocimiento; §no-repudio: por qué los MAC no lo dan).

Qué necesitás saber.

- MAC/HMAC:** terna (Gen, Mac, Vrfy) con clave **compartida** k ; da integridad y autenticación de origen *entre partes que comparten la clave*, pero **no** no-repudio.
- Función de hash:** sin clave, de tamaño fijo, con tres resistencias (preimagen, segunda preimagen, colisión). La resistencia a preimagen es la base de los **protocolos de prueba de conocimiento / compromiso**: publicar $H(x)$ y después revelar x , de forma que nadie pudo deducir x antes de la revelación, y quien reveló x no puede haber cambiado de valor sin que se note (segunda preimagen).
- Ataque de cumpleaños / brute-force de espacios chicos:** si el espacio de posibles mensajes es chico (acá, 2^{55} cartones), un compromiso $H(\text{cartón})$ *sin sal* es vulnerable a que

cualquiera enumere los 2^{55} cartones posibles, los hashee y compare contra el compromiso publicado para deducir el cartón antes de tiempo. La nota del enunciado es justamente el gancho para exigir un **nonce/sal grande** (mucho mayor a 55 bits) dentro del compromiso.

Notas y conexiones. Los tres incisos son variaciones del mismo patrón: (a) y (b) son **compromisos** (commitments) usando la resistencia a preimagen/segunda preimagen de una función de hash (no llevan clave, pero son “funciones criptográficas simétricas” en el sentido amplio que exige la consigna, ya que no dependen de un par de claves pública/privada); (c) es un esquema de **autenticación con clave compartida** (MAC/HMAC) para que todas las apps puedan validar de forma independiente los mismos mensajes del servidor. Como está prohibido usar asimétrica, no hay firma digital disponible, así que ninguna de las tres soluciones da no-repudio frente al servidor (el servidor y las apps comparten secretos).

Resolución. **a) Sorteo verificable.** Antes de arrancar la partida el servidor elige una **semilla secreta** s (número aleatorio, e.g. 256 bits) y publica un **compromiso** $c = H(s)$ a todas las apps (usando la resistencia a preimagen de un hash: nadie puede deducir s a partir de c). La secuencia de 90 números se deriva de forma **determinística** de s con una función pseudoaleatoria con clave (p.ej. $\text{HMAC}_s(i)$ para $i = 1, 2, \dots$, usada como fuente de aleatoriedad de un *shuffle* tipo Fisher–Yates sobre $\{1, \dots, 90\}$). El servidor va revelando los números uno a uno a medida que los “saca”; como se derivan de s que solo él conoce, no puede ser forzado a revelar otro número, pero tampoco puede cambiar el orden a mitad de partida sin que se note (porque ya se comprometió a $c = H(s)$ antes de empezar). Al finalizar la partida, el servidor revela s ; cada app recalcula $H(s)$ y verifica que coincide con c (garantiza que s es el mismo que se comprometió al inicio, por resistencia a segunda preimagen), y recalcula la secuencia completa de 90 números con s para verificar que coincide con la secuencia efectivamente anunciada durante el juego. Si alguna etapa no coincide, la secuencia fue manipulada.

b) Cartón no modificable. Al empezar la partida, cada app cliente genera su cartón y envía al servidor un **compromiso** $h = H(\text{cartón} \parallel r)$, donde r es un **nonce aleatorio grande** (mucho mayor a 55 bits, p.ej. 128 bits) generado por la app y que **no** se envía al servidor en ese momento. Por la nota del enunciado, si se omitiera r el compromiso $H(\text{cartón})$ sería vulnerable a que cualquiera (el propio servidor u otro atacante con el compromiso) recorra los 2^{55} cartones posibles, hasheándolos hasta encontrar el que coincide, revelando el cartón antes de que termine la partida. Agregar r (con un espacio mucho mayor que 2^{55}) hace que la búsqueda por fuerza bruta sea inviable (2^{55+128} combinaciones), preservando la *ocultación* del cartón hasta el final. Al momento de reclamar bingo (o al finalizar la partida), la app envía $(\text{cartón}, r)$ al servidor, que recalcula $H(\text{cartón} \parallel r)$ y verifica que coincide con el h recibido al inicio (resistencia a segunda preimagen: no puede existir otro cartón' con el mismo h , así que el jugador no pudo cambiar el cartón después de comprometerse).

c) Verificación distribuida sin comunicación entre apps. Todas las apps comparten con el servidor (instalada de antemano, al registrarse) una **clave simétrica** k (puede ser una por partida, distribuida al inicio junto con (G, q, g) ... o más simple, generada por el servidor y enviada cifrada/instalada a cada app al arrancar la partida). Cada número que el servidor “saca” en el paso (a) se transmite junto con una etiqueta $t_i = \text{HMAC}_k(i \parallel \text{número}_i)$; cada app, al recibirlo, valida Vrfy_k y solo si es válido lo marca como “anunciado”. De esta forma todas las apps arman localmente el mismo historial **autenticado** de números sacados, sin hablar entre sí (solo escuchan al servidor). Cuando alguien grita bingo, el servidor retransmite a todas las apps el cartón revelado del ganador junto con (r, h) del punto (b); cada app, de forma independiente, (i) recalcula $H(\text{cartón} \parallel r)$ y compara con el h que el servidor había repartido al inicio de la partida (si no coincide, el cartón fue alterado), y (ii) chequea que los 15 números del cartón estén todos entre los que su propio historial local marcó como “anunciados y autenticados con k ”. Si cualquiera de las dos verificaciones falla, la app emite una alerta al servidor (sigue sin comunicarse directamente con las demás apps, tal como pide la consigna).

△ Cuidado — error típico

Ninguna de las tres soluciones da **no-repudio**: como todo se basa en hash sin clave o en un secreto **compartido** (s, r, k) , el servidor sigue siendo, en última instancia, quien controla la información y a quien hay que confiarle la ejecución honesta del protocolo (el enunciado no permite usar firma digital, que sería la única forma de dar no-repudio, según Clase 4).

6.2 Ejercicio — Robo de cables eléctricos

Consigna (textual)

El robo de cables por el valor intrínseco del cobre que traen es un gran problema para las compañías de distribución eléctrica, dado que les genera un doble perjuicio: el costo económico de reponer los cables y el costo asociado a resarcir a clientes cuyo servicio se ve afectado.

Para detectar cortes (intencionales o no) en las líneas, cada cierta distancia se conectan pequeños dispositivos que modulan una señal de alta frecuencia en la misma línea que transporta energía. Cada dispositivo tiene una firma única (una frecuencia y un periodo dado — diseñado de tal forma que todos los dispositivos pueden inyectar su señal sin interferirse entre sí). Las líneas terminan en transformadores. Antes del transformador se coloca un dispositivo llamado *colector* que recolecta todas las señales y aplica un filtro de alta frecuencia que limpia la señal para que llegue al transformador sin este ruido.

Por la forma en que se juntan las señales, cada dispositivo es capaz de transmitir el equivalente a una secuencia de 320 bits arbitraria.

Una implementación en particular de este sistema asigna a cada dispositivo un par de claves pública y privada de 320 bits, y hace que cada dispositivo envíe regularmente la firma digital de su número de serie único (solo la firma!).

El *colector* conoce los números de serie de todos los dispositivos, y de cada uno la frecuencia (lo que le permite extraer los 320 bits de la señal específica), la ubicación relativa en el cable y su clave pública. De esta manera, puede determinar que si deja de recibir la señal del dispositivo x y todos los anteriores hubo un corte en el cable, mientras que si solo falta x hubo un problema con el dispositivo en particular.

- a) Explicar paso a paso que tiene que hacer el colector para validar el estado de la línea y llegar a uno de tres estados: todo ok, corte cercano al dispositivo x , problemas con el dispositivo x . Ser preciso en la aplicación de cada función criptográfica involucrada y sus parámetros. (1 pto)
- b) En busca de abaratar costos, se probó una modificación al sistema donde todos los dispositivos comparten una única clave secreta con el aparato terminal (aunque cada uno sigue teniendo un número de serie diferente), y transmiten una etiqueta de su número de serie, generada con un MAC criptográficamente seguro. ¿Como cambia el algoritmo del colector? ¿Que desventajas genera este cambio a nivel seguridad? (1 pto)
- b) ¿Como modificaría el protocolo para evitar un ataque donde alguien corte una sección pero agregue un aparato que repita las señales enviadas previamente por los dispositivos que quedaron fuera de línea? (1 pto)

Nota: es imposible modificar el ancho de banda de cada dispositivo de 320 bits.

Temas. Clase 4 — Cifrado Asimétrico y Firma Digital (§Firmas digitales; §RSA-Signature y por qué se firma el hash; §curvas elípticas: 320 bits $\approx$ seguridad de RSA-2048); Clase 3 — MACs y Cifrado Autenticado (§Qué es un MAC; §no-repudio: por qué los MAC no lo dan); Clase 5 — Protocolos (§nonce y timestamp como mecanismos de frescura; §ataques de replay y sus defensas).

Qué necesitas saber.

- **Firma digital:** $\text{Sign}_{sk}(m)$ con la clave **privada** del firmante; $\text{Vrfy}_{pk}(m, \sigma)$ con la clave **pública**, que cualquiera (incluido el colector) puede ejecutar sin compartir secretos. Da no-repudio e integridad, y es **públicamente verificable**: el colector solo necesita la clave pública de cada dispositivo, no un secreto compartido.
- **320 bits como firma sobre curva elíptica:** el material de la clase indica que para una seguridad equivalente a RSA-2048 (~ 128 bits) alcanza con ~ 320 bits en curvas elípticas (ECDSA/ElGamal sobre curva). Esto encaja exactamente con el ancho de banda de 320 bits que puede transmitir cada dispositivo: es coherente con que la “firma digital de su número de serie” quepa en una única transmisión.
- **MAC con clave compartida entre todos:** da integridad/autenticación pero **no** distingue quién generó la etiqueta (cualquiera con la clave puede forjar la de cualquier otro) y **no** da no-repudio (Clase 3).
- **Frescura (nonce/timestamp) contra replay:** Clase 5 muestra que sin un mecanismo de frescura, un atacante puede grabar y reenviar (*replay*) un mensaje válido viejo. La defensa es incluir un **nonce** o **timestamp** en lo que se firma/etiqueta, de forma que el verificador rechace mensajes “viejos”.

Notas y conexiones. El enunciado arma un caso de uso real de firma digital asimétrica (Clase 4) con la restricción de ancho de banda de 320 bits, que **calza justo** con el tamaño de firma de un esquema sobre curva elíptica visto en la Clase 4. La parte (b) reproduce el contraste MAC vs. firma digital de la Clase 3 (no-repudio) y la parte (b) repetida conecta con los mecanismos de frescura/anti-replay de la Clase 5 (Needham–Schroeder, Denning–Sacco), pero con la restricción extra de que el tamaño transmitido no puede crecer: se resuelve firmando/etiquetando el **hash** de (serie || contador), tal como en Clase 4 se explica por qué se firma el hash y no el mensaje (permite mensajes de tamaño arbitrario con una firma de tamaño fijo).

Resolución. a) **Algoritmo del colector (esquema original, con firma digital).** El colector mantiene, para cada dispositivo $i = 1, \dots, n$ ordenado por **ubicación relativa en el cable** (desde el más cercano al colector hasta el más lejano), su número de serie serie_i , su frecuencia/periodo (para poder sintonizar y extraer los 320 bits que le corresponden) y su clave pública pk_i . Paso a paso:

1. Para cada dispositivo i , el colector **sintoniza** su frecuencia/periodo específica y trata de capturar los 320 bits transmitidos en esa señal.
2. Si capta señal en la frecuencia de i : interpreta esos 320 bits como una firma σ_i y ejecuta $\text{Vrfy}_{pk_i}(\text{serie}_i, \sigma_i)$. Si la verificación da 1, el dispositivo i está **transmitiendo correctamente** (nadie puede haber falsificado esa firma sin la clave privada de i).
3. Si **no** capta señal en la frecuencia de un dispositivo x y tampoco en la de **todos** los dispositivos ubicados a partir de x hacia el otro extremo del cable (los que están “detrás” de x respecto del colector): como la señal viaja por el mismo conductor de cobre, un corte físico en el punto donde está x interrumpe la continuidad eléctrica para todos los dispositivos más allá del corte. Estado: **corte cercano al dispositivo x** .

4. Si **solo falta** la señal de x , pero los dispositivos ubicados antes y después de x en el cable sí llegan (y verifican $Vrly = 1$): el cable sigue eléctricamente continuo (si no, tampoco llegarían los de más allá de x), así que la falla es puntual del propio dispositivo x (rotura, batería, etc.). Estado: **problema con el dispositivo x** .

5. Si todos los dispositivos verifican correctamente: estado **todo ok**.

b) Variante con clave secreta única (MAC compartido). El algoritmo del colector cambia únicamente en el paso de verificación: en vez de $Vrly_{pk_i}$ con una clave pública distinta por dispositivo, el colector (que ahora debe conocer la **única** clave secreta K compartida por todos los dispositivos) recibe la etiqueta $t_i = MAC_K(\text{serie}_i)$ transmitida en los 320 bits, recalcula $MAC_K(\text{serie}_i)$ y compara con t_i (en vez de verificar una firma con clave pública, recalcula y compara, como cualquier MAC). El resto de la lógica (corte cercano a x vs. problema puntual en x , según qué señales llegan) no cambia.

Desventajas de seguridad:

- **Punto único de falla:** al ser una clave **compartida por todos los dispositivos**, si un atacante extrae físicamente K de *un solo* dispositivo (por ejemplo abriéndolo), **compromete a todo el sistema**: puede generar etiquetas válidas para cualquier número de serie, falsificando la señal de cualquier dispositivo (incluso de los que nunca tocó).
- **Sin no-repudio ni atribución:** como todos comparten K , la etiqueta **no prueba que la transmitió ese dispositivo en particular**: en principio cualquiera que conozca K (incluido otro dispositivo legítimo, o el propio colector) podría haber generado esa etiqueta. Con firma digital (parte a) cada dispositivo es el único que puede producir su firma (clave privada propia); con MAC compartido esa propiedad se pierde por completo.
- El colector mismo pasa a ser un blanco crítico: si se compromete, se filtra K y cae la seguridad de *todos* los dispositivos de la red, no solo de uno.

b) (repetida) — Evitar el replay de señales viejas. El ataque descrito es un **replay**: alguien corta una sección y coloca un aparato que retransmite las firmas/etiquetas **viejas** capturadas de los dispositivos que quedaron desconectados, haciendo que el colector siga viendo “todo ok” aunque esos dispositivos ya no estén. La defensa (Clase 5) es agregar **frescura** (nonce o timestamp/contador) a lo que se firma o etiqueta. Como el ancho de banda está fijado en 320 bits y no se puede agrandar, no alcanza con agregar el contador “en claro” junto a la firma (ocuparía más bits); la solución es la misma idea que justifica firmar el **hash** en la Clase 4: en vez de firmar/etiquetar serie_i solo, cada dispositivo firma/etiqueta $H(\text{serie}_i \parallel c_i)$, donde c_i es un **contador** (o ventana de tiempo) que el dispositivo incrementa en cada transmisión y que el colector puede predecir (ambos, dispositivo y colector, mantienen el contador sincronizado, o el colector acepta una ventana de contadores cercana al último visto). Como el hash tiene salida de tamaño fijo, la firma/etiqueta sigue ocupando exactamente 320 bits, sin importar que ahora el mensaje firmado sea más largo. El colector, al verificar, recalcula $H(\text{serie}_i \parallel c_i^{\text{esperado}})$ y ejecuta $Vrly$ (o recomputa el MAC) contra ese valor; si el aparato del atacante repite una firma/etiqueta vieja, el contador que usó para generarla ya **no coincide** con el esperado por el colector, y la verificación falla (detecta el replay). Si el contador nunca avanza mientras se supone que el dispositivo sigue transmitiendo, el colector puede además usar eso como señal adicional de anomalía.

6.3 Ejercicio — Organizame (pentesting de una app de calendario)

Consigna (textual)

“Organizame” es una aplicación móvil que busca potenciar el uso eficiente del tiempo de las personas.

La aplicación permite que un usuario se registre utilizando su cuenta de Google o su Apple Id, y pide acceso al calendario de dicha cuenta para obtener información de las reuniones agendadas.

La información de agenda es almacenada en una base de datos en el servidor, donde procesos corren regularmente análisis y proponen mejoras a la agenda. Cuando uno de estos procesos encuentra una posible mejora, envía una notificación a la aplicación para que el usuario decida si desea aplicar el cambio o no. Algunos procesos son simples y rápidos, aunque hay algunos que tienen un orden de complejidad $O(n^3)$ en la cantidad de eventos del mes.

La aplicación permite compartir huecos de tiempo libre del calendario de forma publica, para que otras personas puedan agendar reuniones. Para ello disponibiliza una pagina web por usuario, de la forma: `https://organizame.io/{user_id}/public/calendar`. Un usuario puede habilitar o deshabilitar dicha página en su configuración de perfil. En ella, cualquier persona puede agendar una reunión en los momentos disponibles, y dejarle al dueño un mensaje que verá en su calendario.

Ante un pedido de realizar un pentest que abarque el sistema, establecer 4 hipótesis de falla **pertinentes** para el caso, basadas en problemas de seguridad en aplicaciones. Explicar **para cada una**, como probarla según la metodología (la acción concreta que ejecutaría para verificar o refutar la hipótesis). (4 puntos, 1 pto cada uno)

Aclaración: No utilizar problemas genéricos como hipótesis de falla. Plantear situaciones y lugares concretos donde podría estar el problema. Por ejemplo, en lugar de decir “puede haber un buffer overflow”, decir “es probable que haya un buffer overflow en la lectura del parámetro x que debe interpretarse como número y puede tener un buffer pequeño asumiendo que el número tiene menos de los dígitos de un long”.

Temas. Clase 6 (Segunda Parte) — Pentesting (§Metodología de hipótesis de falla, en particular Paso 2 “Hipótesis” y Paso 3 “Prueba”); Clase 1 — Políticas y Control de Acceso (§OAuth 2.0 y *scope*); Clase 3 — Principios de Diseño y Vulnerabilidades (§Tipos de vulnerabilidades: relaciones de confianza, injection flaws; §disponibilidad y DoS como ataque de seguridad).

Qué necesitás saber.

- **Metodología de hipótesis de falla (memorizar):** Recolección de información → Plan-teo de hipótesis de falla → Prueba de la vulnerabilidad → Generalización → Eliminación (opcional). Acá se pide directamente el paso 2 (hipótesis) justificado con el paso 3 (cómo probarla): “definir cómo probar” prioriza *analizar documentación o comportamiento* antes que intentar explotar, y la prueba debe ser **repetible y concluyente**.
- **Relaciones de confianza (Clase 3, tipos de vulnerabilidades):** “no chequear autorización porque *nadie va a adivinar la URL*” es exactamente el patrón de la URL `/ {user_id} /public/calenda` hay que verificar que el servidor efectivamente valide el flag de “habilitado” del usuario y no simplemente confíe en que el `user_id` es difícil de adivinar.
- **OAuth y scope (Clase 1):** el token de acceso debe limitarse al *scope* mostrado en la pantalla de consentimiento (acá, lectura de reuniones); pedir de más (por ejemplo, permiso de escritura/administración total del calendario) es una falla de diseño de autorización.

- **Disponibilidad como propiedad de seguridad (Clase 3):** un ataque que tira o degrada el sistema (DoS) es un ataque de seguridad aunque no comprometa confidencialidad ni integridad; un proceso de complejidad $O(n^3)$ sobre un dato que un atacante puede inflar (cantidad de eventos del mes) es candidato directo.
- **Injection flaws (Clase 3):** el profesor remarca que $\sim 90\%$ de las vulnerabilidades caen en esta familia; cualquier campo de entrada no controlado (acá, el “mensaje” que deja un desconocido en el calendario público) es un lugar concreto donde buscarla.

Notas y conexiones. Las cuatro hipótesis se ubican, cada una, en un componente concreto descrito en el enunciado (la URL pública, el proceso $O(n^3)$, el flujo OAuth con Google/Apple, el campo de mensaje libre), tal como pide la aclaración de no usar hipótesis genéricas. Todas están tomadas de puntos ya vistos en el material (relaciones de confianza, disponibilidad/DoS, scope de OAuth, injection flaws) y no se agrega nada fuera de eso.

Resolución. Hipótesis 1 — IDOR / relación de confianza en `/user_id/public/calendar`.

Es probable que el servidor, al recibir un GET a esa URL, no vuelva a validar que el usuario efectivamente tenga habilitada la opción “pública” en su perfil, confiando en que `user_id` es difícil de adivinar o de enumerar (la relación de confianza “nadie va a adivinar la URL” que señala el material). Si los `user_id` son secuenciales o de algún modo predecibles, cualquiera podría ver/agendar en calendarios que el dueño nunca hizo públicos. *Cómo probarla:* crear una cuenta de prueba con la página pública **deshabilitada**, y con otra cuenta (o sin sesión) acceder directamente a `https://organizame.io/{ese_user_id}/public/calendar`; si igual devuelve datos de agenda o permite agendar, queda demostrado. También probar con `user_id` consecutivos a uno propio habilitado, para ver si expone calendarios de terceros que no lo sabían.

Hipótesis 2 — Denegación de servicio vía el proceso $O(n^3)$. Es probable que el proceso de análisis de complejidad $O(n^3)$ en la cantidad de eventos del mes no tenga un límite de eventos por usuario/mes, por lo que un usuario (o un atacante que agenda a través de la página pública de un tercero) podría inflar artificialmente la cantidad de eventos de un mes para que ese proceso batch consuma recursos desproporcionados, degradando la disponibilidad del servicio para otros usuarios. *Cómo probarla:* con una cuenta de prueba, generar mediante un script una gran cantidad de eventos en un mismo mes (por API propia o a través del flujo de agendar reunión desde una página pública), y medir el tiempo/CPU que insume la corrida del proceso de análisis para ese mes, verificando si crece de forma cúbica y si llega a afectar al resto del sistema (colas, timeouts, caída del servicio).

Hipótesis 3 — Scope de OAuth excesivo al integrar Google/Apple Calendar. Es probable que, al pedir “acceso al calendario” de la cuenta de Google/Apple, la app solicite un *scope* más amplio del necesario (por ejemplo, lectura y escritura/administración del calendario, o acceso a otros datos de la cuenta), cuando para leer reuniones agendadas alcanzaría con un scope de solo lectura. *Cómo probarla:* iniciar el flujo de registro con Google/Apple y revisar la pantalla de consentimiento y/o decodificar el *access token* obtenido, verificando qué scopes concretos se solicitan y comparándolos con lo mínimo necesario (leer eventos); si el scope permite modificar o borrar eventos de la cuenta real de Google/Apple del usuario sin que eso sea necesario para la funcionalidad descrita, queda demostrado el exceso.

Hipótesis 4 — Injection en el campo de mensaje de la página pública. Es probable que el campo de texto donde un desconocido deja un mensaje al agendar una reunión desde `/user_id/public/calendar` no sanitice la entrada antes de guardarla o de mostrarla en el calendario del dueño, permitiendo una inyección (por ejemplo, un payload de script si el mensaje se renderiza en la vista web del calendario del dueño). *Cómo probarla:* desde la página pública, agendar una reunión de prueba dejando como mensaje un payload típico de prueba (por ejemplo una etiqueta de script inocua, o una comilla simple si el mensaje se guarda en una base de datos

SQL), y luego revisar, desde la cuenta del dueño, si el mensaje se ejecuta/se interpreta en vez de mostrarse como texto plano, o si genera un error de base de datos que delate una inyección.

△ Cuidado — error típico

No se incluyen hipótesis genéricas (“puede haber XSS en algún lado”, “puede haber SQL injection en algún formulario”) porque la consigna explícitamente lo prohíbe: cada hipótesis de arriba está anclada a un componente y un flujo concreto descrito en el enunciado de “Organizame”.